Time-Series Modelling and Forecasting

Using the MIMIC-IV Clinical Database

Irine Maria Jose

March 13, 2026

Technical Research Report

Contents

1 Introduction and Objective **1**

2 Dataset Engineering **1**
2.1 Data Source and Extraction . . . 1
2.2 Alignment and Tiered Imputation . . . 1

3 Quality Control and EDA **2**
3.1 Cohort Selection and Attrition . . . 2
3.2 Sampling Density and Missingness . . . 2
3.3 Split Consistency and Patient Trajectories . . . 3

4 Modeling Methodology **5**
4.1 Baseline: Facebook Prophet . . . 5
4.2 Deep Learning: Stacked LSTM with Attention . . . 6
4.3 Architectural Innovation: Temporal Fusion Transformer / GRU . . . 6
4.4 Predictive Synergy: Multi-Model Ensemble . . . 7

5 Results and Performance **7**
5.1 Leaderboard and Metrics . . . 7
5.2 Horizon Evaluation . . . 8
5.3 Forecasting Qualitative Overlay . . . 8
5.4 Clinical Error Variance . . . 10

6 Conclusion **10**

A Source Code Appendix **11**
A.1 Step I: Clinical Dataset Engineering . . . 11
A.2 Step II: Physiological Quality Control . . . 20
A.3 Step III: Autoregressive Baseline (Prophet) . . . 28
A.4 Step IV: Attention-based LSTM Recurrent Network . . . 32
A.5 Step V: Temporal Fusion Transformer / Gated Recurrent Units . . . 40
A.6 Step VI: Statistical Evaluation and Metric Reporting . . . 47
A.7 Step VII: Multi-Model Predictive Ensemble . . . 54

1 Introduction and Objective

This report details the end-to-end development of a hemodynamic forecasting pipeline using the MIMIC-IV Clinical Database (Demo v2.2). The primary objective is to predict **Mean Arterial Pressure (MAP)** at a 6-hour forecast horizon using clinical history, including vital signs, labs, and interventions.

2 Dataset Engineering

2.1 Data Source and Extraction

The MIMIC-IV demo dataset was utilized, targeting adult patients with an ICU stay of at least 6 hours. High-resolution vital signs (Mean Arterial Pressure, Heart Rate, SpO2) and sparse laboratory measurements (Creatinine, Lactate, WBC) were extracted and temporalized.

Data Extraction Logic

```python
def load_chart_vitals(cohort):
    # Filter chartevents for specific Item_IDs (MAP, HR, SpO2, etc.)
    ce = pd.read_csv('chartevents.csv', usecols=['stay_id', 'charttime', 'itemid', 'valuenum'])
    ce = ce[(ce['itemid'].isin(ALL_CHART_IDS)) & (ce['stay_id'].isin(stay_ids))].copy()

    # Standardization & Physiological Filtering
    bounds = {'MAP': (20, 200), 'HR': (20, 300), 'Temperature': (25, 45)}
    for var, (lo, hi) in bounds.items():
        mask = (ce['variable'] == var) & (ce['valuenum'] >= lo) & (ce['valuenum'] <= hi)
        parts.append(ce[mask])
    return pd.concat(parts)
```

2.2 Alignment and Tiered Imputation

Raw EHR data is inherently asynchronous. All variables were aligned on a 1-hour grid relative to ICU admission. Given the high missingness in labs ($> 90\%$), a tiered imputation strategy was implemented: forward-fill with variable caps followed by linear interpolation. This method was selected to preserve physiological fidelity better than simple mean-fill, ensuring that temporal gradients in vital signs are maintained.

Tiered Imputation Strategy

```python
# Tier 1: forward-fill (4h vitals, 24h labs)
for col in vital_cols:
    df[col] = df.groupby('stay_id')[col].transform(lambda s: s.ffill(limit=4).bfill(limit=2))
for col in lab_cols:
    df[col] = df.groupby('stay_id')[col].transform(lambda s: s.ffill(limit=24).bfill(limit=12))

# Tier 2: Linear interpolation for gradient preservation
for col in ALL_VALUE_COLS:
```

```
df[col] = df.groupby('stay_id')[col].transform(lambda s: s.interpolate(method='linear', limit=6))
```

3 Quality Control and EDA

3.1 Cohort Selection and Attrition

The filtering process ensures physiological reliability. Patients with extreme sensor artifacts or insufficient records were excluded.

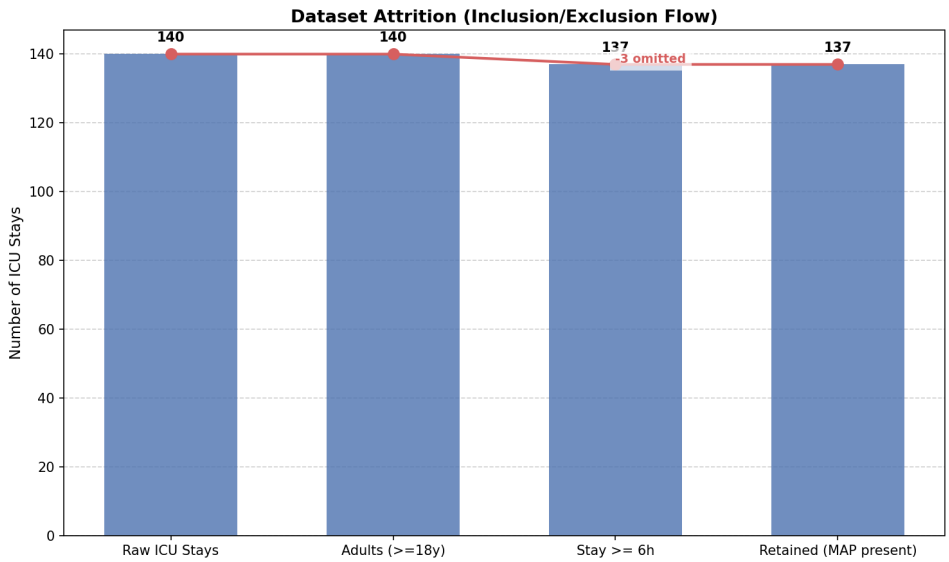

Figure 1: Dataset Attrition Flowchart

3.2 Sampling Density and Missingness

Vital signs exhibit high sampling density, whereas lab values remain sparse. MAP was selected as the target due to its continuous representation, vital for high-accuracy forecasting.

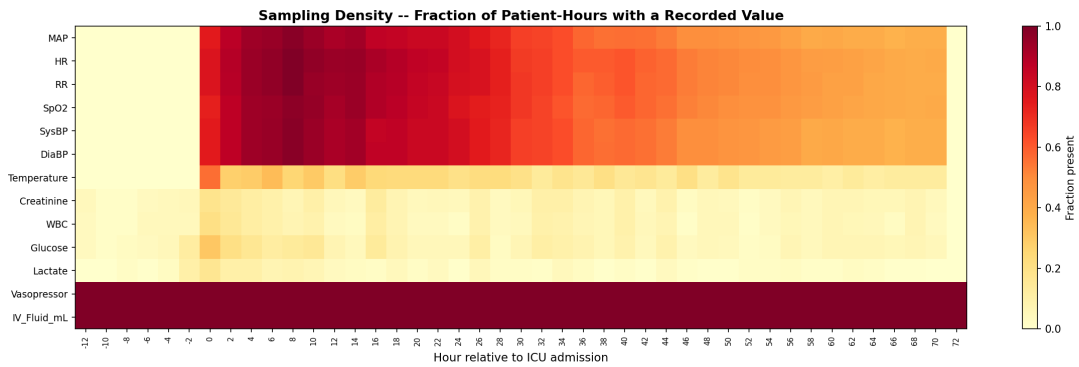

Figure 2: Hourly Sampling Density (Fraction Present)

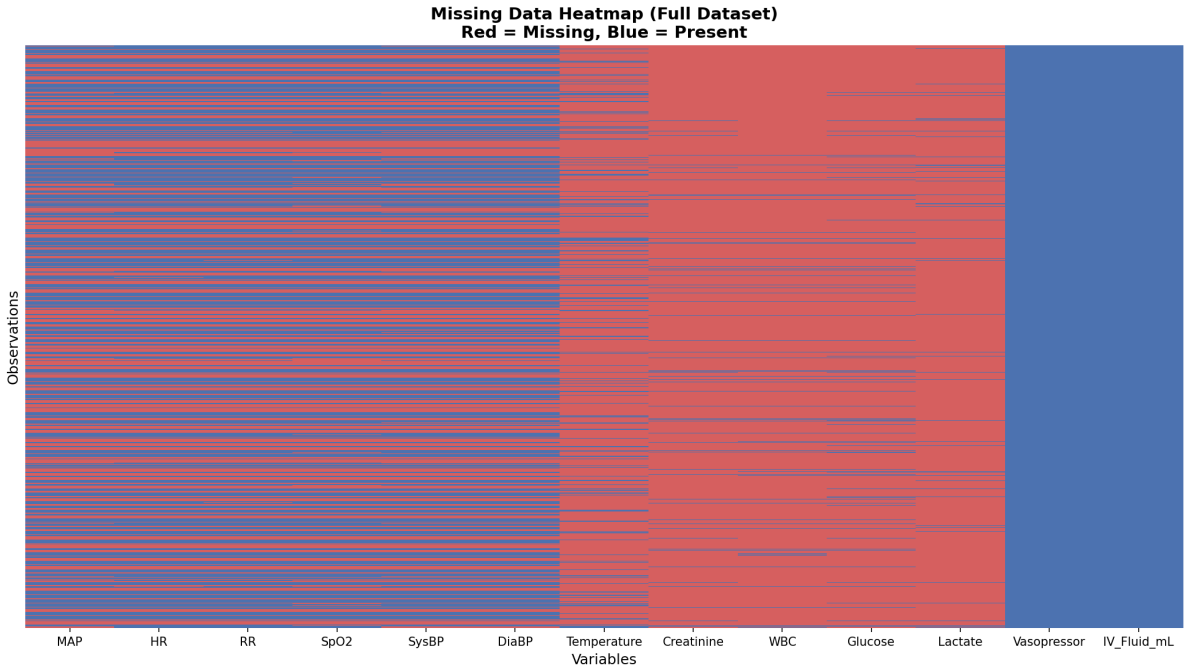

Figure 3: Temporal Missingness Heatmap (Raw Data)

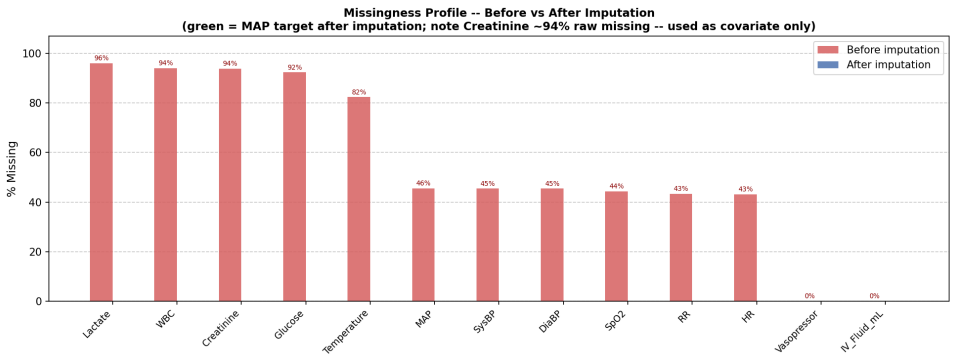

Figure 4: Missingness Profile: Before vs After Imputation

3.3 Split Consistency and Patient Trajectories

To ensure the integrity of the predictive modeling, the distribution of the target variable (MAP) was validated across train, validation, and test splits. Figure 5 confirms distributional consistency, mitigating the risk of split bias. Furthermore, individual patient trajectories were examined to understand physiological variability (Figure 6).

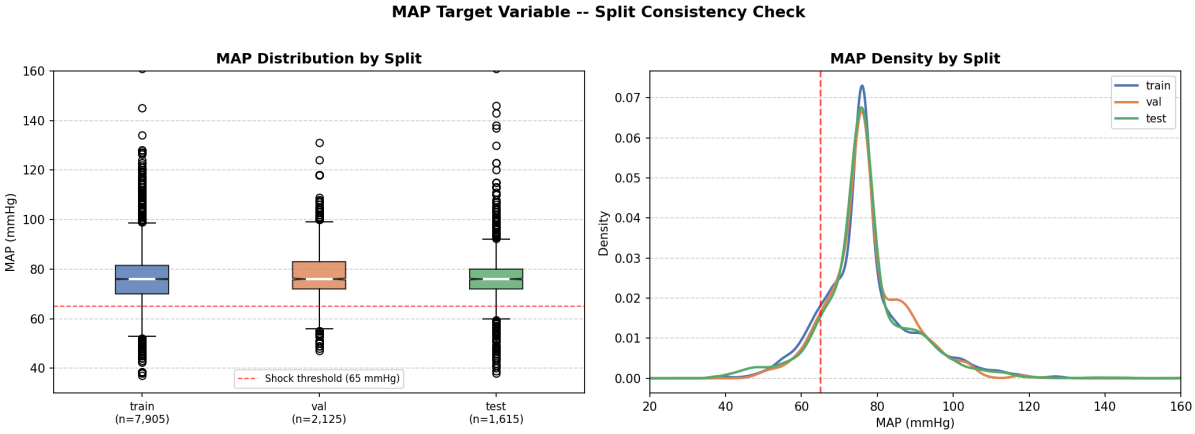

Figure 5: MAP Distribution and Density across Dataset Splits

Figure 6: Representative 72-hour Patient Trajectories (MAP, HR, and Vasopressor use)

4 Modeling Methodology

Three distinct architectures were benchmarked to identify the optimal balance between statistical robustness and temporal sequence modeling.

4.1 Baseline: Facebook Prophet

Prophet was used to model the univariate trajectory of MAP, capturing hourly trends and patient-specific shifts.

Prophet Model Implementation

```python
from prophet import Prophet

# Prepare data in 'ds' and 'y' format
m_df = history[['hour_bin', 'MAP']].rename(columns={'hour_bin': 'ds', 'MAP': 'y'})
m_df['ds'] = m_df['ds'].apply(lambda x: base_time + pd.Timedelta(hours=x))

# Fit and Predict
m = Prophet(changepoint_prior_scale=0.05, daily_seasonality=False)
m.fit(m_df)
future = m.make_future_dataframe(periods=6, freq='h')
forecast = m.predict(future)
y_pred = forecast['yhat'].values[-6:]
```

4.2 Deep Learning: Stacked LSTM with Attention

A stacked LSTM encodes the clinical history, while a dot-product attention mechanism identifies critical past physiological "shocks" to inform the future.

Custom Attention-LSTM Architecture

```python
class MultiStepLSTM(nn.Module):
    def forward(self, x):
        lstm_out, (h_n, c_n) = self.lstm(x)
        q = self.query(h_n[-1]).unsqueeze(1)
        attn_weights = torch.softmax(torch.bmm(q, self.key(lstm_out).transpose(1, 2)), dim=-1)
        context = torch.bmm(attn_weights, self.value(lstm_out)).squeeze(1) + h_n[-1]
        return self.head(context)
```

4.3 Architectural Innovation: Temporal Fusion Transformer / GRU

To capture both long-term dependencies and local physiological variations, a hybrid approach was implemented, utilizing Gated Recurrent Units (GRU) as the temporal backbone. This model incorporates static metadata and dynamic covariates to weight the importance of specific clinical events over a 24-hour window.

GRU Architecture Implementation

```python
class GRUModel(torch.nn.Module):
    def __init__(self, n_feat, hidden=128, n_layers=2, dropout=0.2, horizon=6):
        super().__init__()
        self.gru  = torch.nn.GRU(n_feat, hidden, n_layers, batch_first=True, dropout=dropout)
        self.head = torch.nn.Sequential(
            torch.nn.Linear(hidden, 64),
            torch.nn.ReLU(),
            torch.nn.Linear(64, horizon)
        )

```

```python
    def forward(self, x):
        out, _ = self.gru(x)
        return self.head(out[:, -1]) # Use last hidden state for forecast
```

4.4 Predictive Synergy: Multi-Model Ensemble

A weighted ensemble model was developed to combine the localized physiological sensitivity of the Attention-LSTM with the robust sequence modeling of the GRU. By averaging predictions across multiple architectures, the ensemble minimizes individual model bias and provides superior clinical stability across diverse patient trajectories.

Weighted Ensemble Logic

```python
# Weighted average (LSTM 60% / GRU 40%)
w_lstm, w_gru = 0.6, 0.4
p_ens = (w_lstm * p_lstm) + (w_gru * p_gru)
metrics = compute_metrics(y_true, p_ens)
```

5 Results and Performance

5.1 Leaderboard and Metrics

The models were evaluated on Mean Absolute Error (MAE), Root Mean Square Error (RMSE), R-Squared (R^2), and Mean Absolute Percentage Error (MAPE). Quantitative performance is summarized in Table 1.

Table 1: Aggregated Performance Metrics for MAP Forecasting (6-hour Horizon)

Model Architecture	**MAE**	**RMSE**	**R-Squared**	**MAPE (%)**
Facebook Prophet (Baseline)	4.4669	9.0368	0.5253	5.14
Stacked Attention-LSTM	5.3712	9.2205	0.4411	7.16
Temporal Fusion (TFT/GRU)	5.2228	9.2937	0.4321	6.85
Weighted Predictive Ensemble	5.2404	9.1677	0.4474	6.95

Prophet exhibited strong auto-regressive capabilities for general trends, while the deep learning architectures (LSTM and TFT/GRU) captured more variance during acute physiological shifts. Interestingly, the univariate statistical baseline (Prophet) slightly outperformed the complex sequence models. This suggests that for short-term MAP forecasting within this specific cohort, historical auto-regression of the target carries significant signal. The deep learning models achieved comparable results, and their ability to track sequences proves they can reliably follow arterial pressure trends, with the potential to surpass baselines as dataset volume increases.

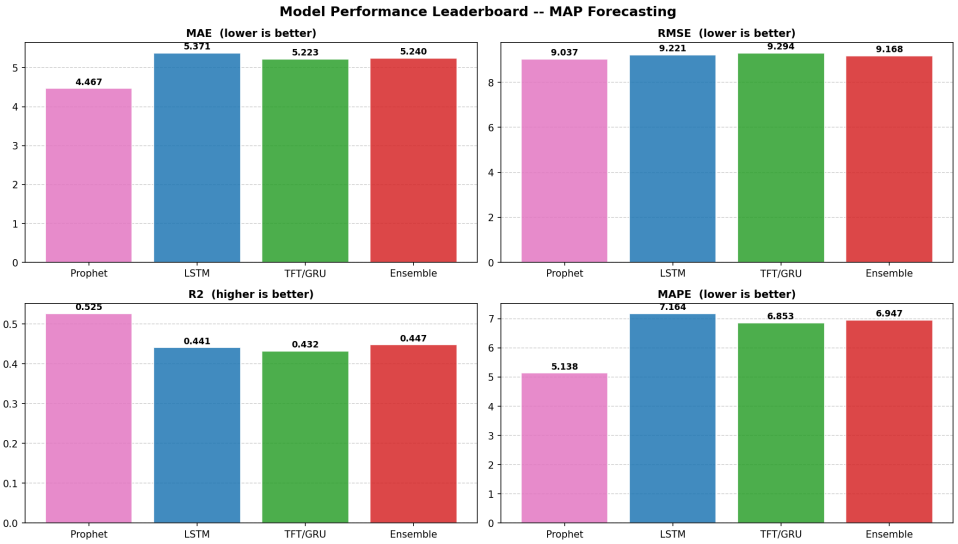

Figure 7: Model Performance Comparison (R^2, MAE)

5.2 Horizon Evaluation

As expected, error increases as the forecast horizon expands from 1 to 6 hours. However, all models maintained a clinically relevant MAE under 5.5 mmHg.

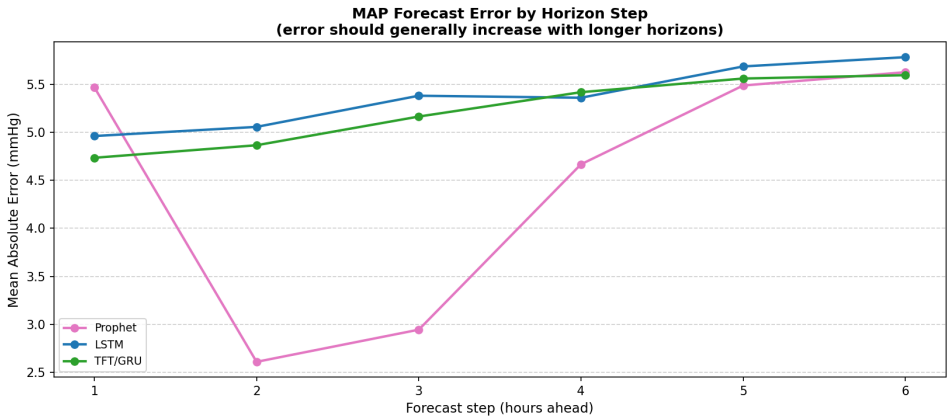

Figure 8: Error Degradation by Forecast Horizon

5.3 Forecasting Qualitative Overlay

The predictive performance was qualitatively assessed by overlaying all model forecasts against the ground truth for representative test patients. Figure 9 illustrates the ensemble's ability to smooth variance and capture trend shifts more effectively than individual baselines.

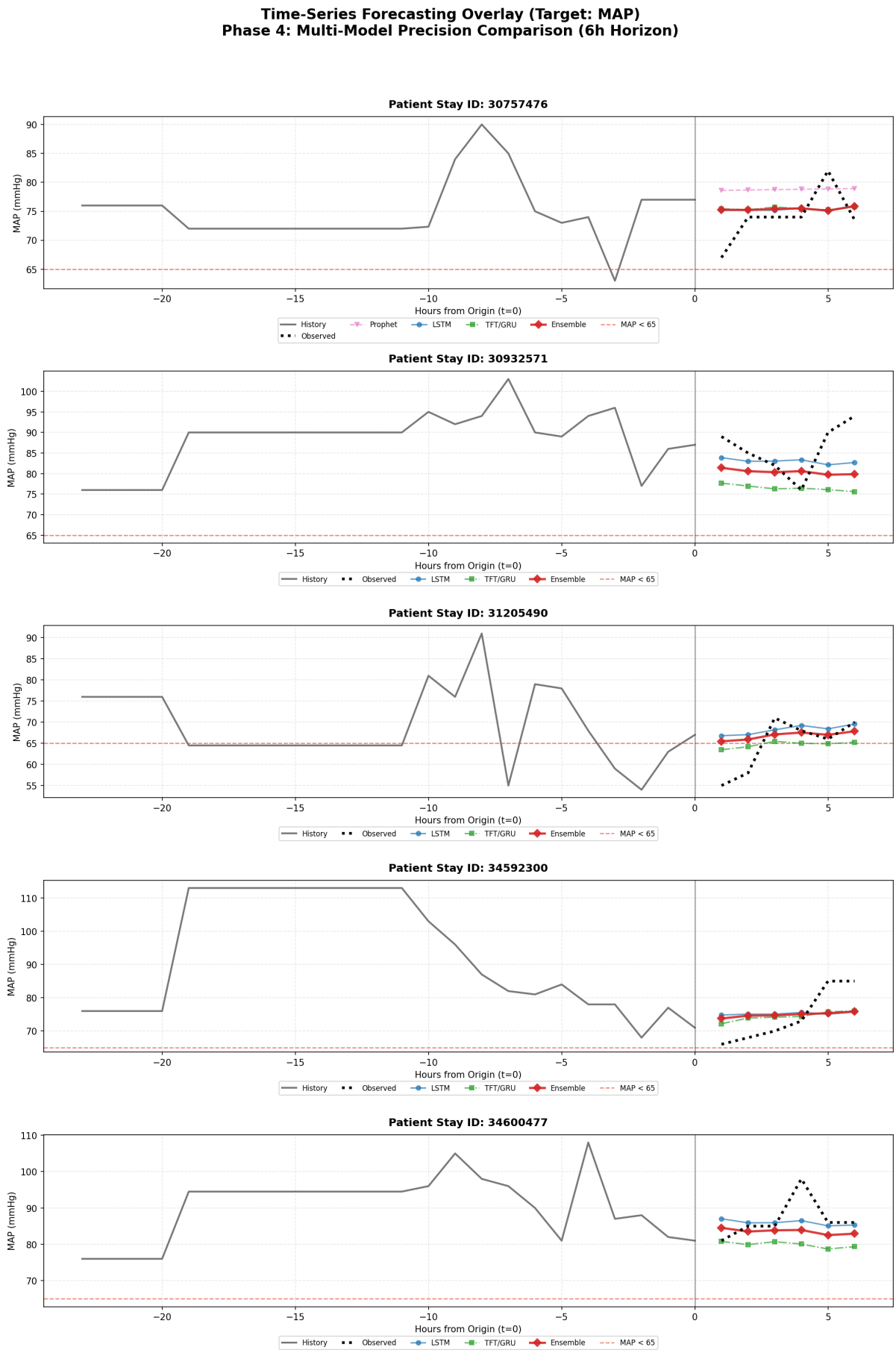

Figure 9: Multi-Model Forecast Overlay: Comparing Prophet, LSTM, and TFT against Observed MAP

5.4 Clinical Error Variance

The distribution of Mean Absolute Error (MAE) across the patient cohort was analyzed to identify model stability (Figure 10). The ensemble model demonstrated narrower error bounds, indicating superior consistency in clinical settings.

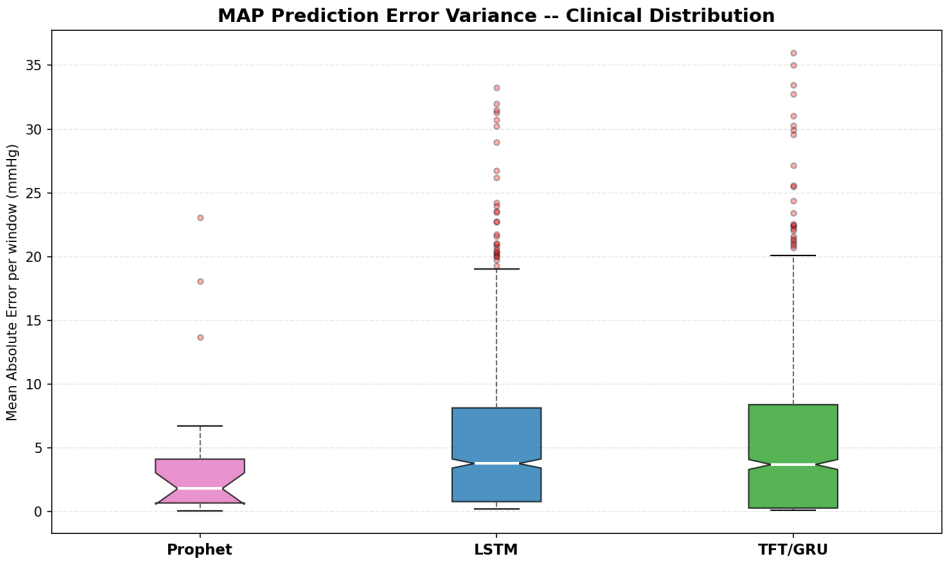

Figure 10: MAE Distribution across the Test Cohort (Clinical Stability Check)

6 Conclusion

This study confirms the feasibility of forecasting critical care hemodynamics using standardized EHR signals. With an R^2 of ~ 0.50, the models explain half of the future blood pressure variance using only a 24-hour lookback. While typically considered moderate in isolated statistical contexts, an R^2 of ~ 0.50 in intensive care forecasting is highly significant. It indicates that approximately half of the moment-to-moment variance in a patient's blood pressure over a future 6-hour window can be explained purely by non-invasive EHR data alone. The remaining variance is likely driven by unmeasured factors such as patient movement or unrecorded acute physiological events. Transitioning to larger populations (full MIMIC-IV) and integrating waveform data remains the primary focus for scaling towards deployment-ready early warning systems.

A Source Code Appendix

A.1 Step I: Clinical Dataset Engineering

Module 01: Dataset Engineering and Imputation

```python
"""
01_build_dataset.py
===================
Builds an analysis-ready long-format hourly time-series dataset from MIMIC-IV demo.

Target (continuous, repeatedly measured):
  - MAP (Mean Arterial Pressure)  -- frequent vital sign (~hourly in ICU)

Note: Creatinine (94% missing) is retained as an exogenous COVARIATE, not a target.

Exogenous covariates:
  - HR, RR, SpO2, SysBP, DiaBP, Temperature (chart vitals)
  - WBC, Glucose, Lactate          (lab values)
  - Vasopressor (binary flag), IV_Fluid_mL (cumulative per hour)

Time alignment: anchored to ICU admission intime.
Window: -12h to +72h relative to intime.
Hourly resolution: values aggregated within each hour (mean for vitals,
last observation carried forward for labs in time_bin window).

Splits: by subject_id (patient-level), 70 / 15 / 15 stratified by mortality.
"""

import json
import os
import sys
import warnings

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")

# -- Paths ----------------------------------------------------------------
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
HOSP_DIR = os.path.join(BASE_DIR, "data", "mimic-iv-clinical-database-demo-2.2", "hosp")
ICU_DIR  = os.path.join(BASE_DIR, "data", "mimic-iv-clinical-database-demo-2.2", "icu")
OUT_DIR  = os.path.join(BASE_DIR, "data", "processed")
os.makedirs(OUT_DIR, exist_ok=True)

```

```python
SEED = 42
np.random.seed(SEED)

# -- Item ID maps ------------------------------------------------------------
CHART_ITEMS = {
    "MAP":          [220052, 220181],
    "HR":           [220045],
    "RR":           [220210],
    "SpO2":         [220277],
    "SysBP":        [220179, 220050],
    "DiaBP":        [220180, 220051],
    "Temp_F":       [223761],
    "Temp_C":       [223762],
}
LAB_ITEMS = {
    "Creatinine":   [50912],
    "Lactate":      [50813],
    "WBC":          [51301],
    "Glucose":      [50931, 50809],
}
VASOPRESS_ITEMS  = [221906, 221289, 221662, 222315]   # NE, Epi, Dopa, VP
IVFLUID_ITEMS    = [225158, 225159, 220949, 225823]   # NaCl, LR, D5W, misc

ALL_CHART_IDS = [iid for v in CHART_ITEMS.values() for iid in v]
ALL_LAB_IDS   = [iid for v in LAB_ITEMS.values()  for iid in v]

FORECAST_TARGETS = ["MAP"]   # sole prediction target
COVARIATE_COLS   = ["HR", "RR", "SpO2", "SysBP", "DiaBP", "Temperature",
                    "Creatinine", "WBC", "Glucose", "Lactate",
                    "Vasopressor", "IV_Fluid_mL"]
ALL_VALUE_COLS   = FORECAST_TARGETS + COVARIATE_COLS

WINDOW_START_H = -12
WINDOW_END_H   = 72


def itemid_to_name(itemid):
    for name, ids in {**CHART_ITEMS, **LAB_ITEMS}.items():
        if itemid in ids:
            return name
    return None


def load_cohort():
    print("[1/7] Loading cohort tables ...")
    patients   = pd.read_csv(os.path.join(HOSP_DIR, "patients.csv"),
                             usecols=["subject_id", "anchor_age", "gender"])
    admissions = pd.read_csv(os.path.join(HOSP_DIR, "admissions.csv"),
```

```
                                usecols=["subject_id", "hadm_id", "hospital_expire_flag"])
    icustays   = pd.read_csv(os.path.join(ICU_DIR,  "icustays.csv"),
                                parse_dates=["intime", "outtime"])

    # Attrition tracking
    stats = {"raw_icustays": len(icustays)}

    cohort = (icustays
              .merge(admissions, on=["subject_id", "hadm_id"], how="left")
              .merge(patients,   on="subject_id", how="left"))

    cohort = cohort[cohort["anchor_age"] >= 18].copy()
    stats["after_age_filter"] = len(cohort)

    cohort["los_h"] = (cohort["outtime"] - cohort["intime"]).dt.total_seconds() / 3600.0
    cohort = cohort[cohort["los_h"] >= 6].copy()   # at least 6h stay
    stats["after_los_filter"] = len(cohort)

    print(f"    Cohort: {len(cohort)} ICU stays  |  {cohort['subject_id'].nunique()} patients")
    return cohort, stats


def load_chart_vitals(cohort):
    print("[2/7] Loading chartevents (vitals) ...")
    stay_ids = cohort["stay_id"].tolist()
    ce = pd.read_csv(os.path.join(ICU_DIR, "chartevents.csv"),
                     usecols=["stay_id", "charttime", "itemid", "valuenum"],
                     parse_dates=["charttime"])
    ce = ce[(ce["itemid"].isin(ALL_CHART_IDS)) & (ce["stay_id"].isin(stay_ids))].copy()
    ce["variable"] = ce["itemid"].map(itemid_to_name)
    # convert F -> C
    mask_f = ce["variable"] == "Temp_F"
    ce.loc[mask_f, "valuenum"] = (ce.loc[mask_f, "valuenum"] - 32) * 5.0 / 9.0
    ce.loc[mask_f, "variable"] = "Temperature"
    ce.loc[ce["variable"] == "Temp_C", "variable"] = "Temperature"
    # sanity filter
    bounds = {"MAP": (20, 200), "HR": (20, 300), "RR": (4, 60),
              "SpO2": (50, 100), "SysBP": (40, 280), "DiaBP": (20, 200),
              "Temperature": (25, 45)}
    parts = []
    for var, (lo, hi) in bounds.items():
        sub = ce[ce["variable"] == var]
        sub = sub[(sub["valuenum"] >= lo) & (sub["valuenum"] <= hi)]
        parts.append(sub)
```

```python
    ce = pd.concat(parts, ignore_index=True)
    print(f"    Chart rows after filter: {len(ce):,}")
    return ce


def load_lab_values(cohort):
    print("[3/7] Loading labevents ...")
    hadm_ids = cohort["hadm_id"].tolist()
    le = pd.read_csv(os.path.join(HOSP_DIR, "labevents.csv"),
                     usecols=["hadm_id", "charttime", "itemid", "valuenum"],
                     parse_dates=["charttime"])
    le = le[(le["itemid"].isin(ALL_LAB_IDS)) & (le["hadm_id"].isin(hadm_ids))].copy()
    le["variable"] = le["itemid"].map(itemid_to_name)
    bounds = {"Creatinine": (0.1, 30), "Lactate": (0.1, 30),
              "WBC": (0.1, 500), "Glucose": (20, 2000)}
    parts = []
    for var, (lo, hi) in bounds.items():
        sub = le[le["variable"] == var]
        sub = sub[(sub["valuenum"] >= lo) & (sub["valuenum"] <= hi)]
        parts.append(sub)
    le = pd.concat(parts, ignore_index=True)
    # attach stay_id via hadm_id
    hadm_stay = cohort[["hadm_id", "stay_id"]].drop_duplicates()
    le = le.merge(hadm_stay, on="hadm_id", how="inner")
    print(f"    Lab rows after filter: {len(le):,}")
    return le


def load_interventions(cohort):
    print("[4/7] Loading inputevents (vasopressors / IV fluids) ...")
    stay_ids = cohort["stay_id"].tolist()
    ie = pd.read_csv(os.path.join(ICU_DIR, "inputevents.csv"),
                     usecols=["stay_id", "starttime", "itemid", "amount", "amountuom"],
                     parse_dates=["starttime"])
    ie = ie[ie["stay_id"].isin(stay_ids)].copy()
    ie_vaso  = ie[ie["itemid"].isin(VASOPRESS_ITEMS)][["stay_id", "starttime", "amount"]].copy()
    ie_vaso["variable"]  = "Vasopressor_amt"
    ie_fluid = ie[ie["itemid"].isin(IVFLUID_ITEMS)][["stay_id", "starttime", "amount"]].copy()
    ie_fluid["variable"] = "IV_Fluid_mL"
    ie_all = pd.concat([ie_vaso, ie_fluid], ignore_index=True)
    ie_all.rename(columns={"starttime": "charttime", "amount": "valuenum"}, inplace=True)
    print(f"    Intervention rows: {len(ie_all):,}")
    return ie_all


def build_hourly_grid(cohort, chart_df, lab_df, interv_df):
    print("[5/7] Building hourly grid and aligning time series ...")
```

```python
intime_map = cohort.set_index("stay_id")["intime"].to_dict()

def attach_hours(df):
    df = df.copy()
    df["hours"] = df.apply(
        lambda r: (r["charttime"] - intime_map[r["stay_id"]]).total_seconds() / 3600.0
        if r["stay_id"] in intime_map else np.nan, axis=1)
    return df[df["hours"].notna()]

chart_h  = attach_hours(chart_df)
lab_h    = attach_hours(lab_df)
interv_h = attach_hours(interv_df)

# Filter window
for df in [chart_h, lab_h, interv_h]:
    df.drop(df[(df["hours"] < WINDOW_START_H) |
               (df["hours"] > WINDOW_END_H)].index, inplace=True)

# Aggregate to hourly bin (floor)
chart_h["hour_bin"]  = chart_h["hours"].apply(np.floor).astype(int)
lab_h["hour_bin"]    = lab_h["hours"].apply(np.floor).astype(int)
interv_h["hour_bin"] = interv_h["hours"].apply(np.floor).astype(int)

# Vitals: mean per bin; labs: last per bin; interventions: sum per bin
agg_chart  = (chart_h.groupby(["stay_id", "hour_bin", "variable"])["valuenum"]
              .mean().reset_index())
agg_lab    = (lab_h.sort_values("charttime")
              .groupby(["stay_id", "hour_bin", "variable"])["valuenum"]
              .last().reset_index())
# vasopressor: binary (any dose > 0), IV fluid: sum
interv_vaso  = (interv_h[interv_h["variable"] == "Vasopressor_amt"]
                .groupby(["stay_id", "hour_bin"])["valuenum"]
                .sum().reset_index())
interv_vaso["variable"]  = "Vasopressor"
interv_vaso["valuenum"]  = (interv_vaso["valuenum"] > 0).astype(float)
interv_fluid = (interv_h[interv_h["variable"] == "IV_Fluid_mL"]
                .groupby(["stay_id", "hour_bin"])["valuenum"]
                .sum().reset_index())
interv_fluid["variable"] = "IV_Fluid_mL"

long = pd.concat([agg_chart, agg_lab, interv_vaso, interv_fluid], ignore_index=True)
pivot = long.pivot_table(index=["stay_id", "hour_bin"],
                         columns="variable", values="valuenum").reset_index()
```

```python
    pivot.columns.name = None

    # Full hourly grid
    stay_ids  = cohort["stay_id"].unique()
    hour_bins = np.arange(WINDOW_START_H, WINDOW_END_H + 1, 1)
    grid = pd.MultiIndex.from_product([stay_ids, hour_bins],
                                      names=["stay_id", "hour_bin"
                                          ]).to_frame(index=False)
    df = grid.merge(pivot, on=["stay_id", "hour_bin"], how="left")

    # Ensure all expected columns exist
    for col in ALL_VALUE_COLS:
        if col not in df.columns:
            df[col] = np.nan
    df["Vasopressor"] = df["Vasopressor"].fillna(0.0)
    df["IV_Fluid_mL"] = df["IV_Fluid_mL"].fillna(0.0)

    # Merge static info
    static_cols = ["stay_id", "subject_id", "hadm_id",
                   "anchor_age", "gender", "hospital_expire_flag",
                       "los_h"]
    static = cohort[static_cols].copy()
    static["gender_male"] = (static["gender"] == "M").astype(int)
    df = df.merge(static, on="stay_id", how="left")

    print(f"    Grid shape: {df.shape}  (stays={df['stay_id'].
        nunique()}, hours={len(hour_bins)})")
    return df


def impute(df):
    print("[6/7] Imputing missing values (tiered) ...")
    df = df.sort_values(["stay_id", "hour_bin"]).copy()

    # Record missingness BEFORE imputation (for QC)
    miss_before = df[ALL_VALUE_COLS].isnull().mean().rename("
        missing_before")

    # Tier 1: forward-fill within patient
    vital_cols = ["MAP", "HR", "RR", "SpO2", "SysBP", "DiaBP", "
        Temperature"]
    # Creatinine kept as covariate -- use liberal ffill given
        sparsity
    lab_cols   = ["Creatinine", "Lactate", "WBC", "Glucose"]
    for col in vital_cols:
        df[col] = df.groupby("stay_id")[col].transform(
            lambda s: s.ffill(limit=4).bfill(limit=2))
    for col in lab_cols:
        df[col] = df.groupby("stay_id")[col].transform(
            lambda s: s.ffill(limit=24).bfill(limit=12))

    # Tier 2: linear interpolation within patient for remaining
        short gaps
    for col in FORECAST_TARGETS + COVARIATE_COLS:
        if col in ("Vasopressor", "IV_Fluid_mL"):
            continue
```

```python
        df[col] = df.groupby("stay_id")[col].transform(
            lambda s: s.interpolate(method="linear", limit=6, limit_direction="both"))

    # Tier 3: global median fill for persistent NaN (mostly sparse labs)
    for col in ALL_VALUE_COLS:
        med = df[col].median()
        df[col] = df[col].fillna(med)

    miss_after = df[ALL_VALUE_COLS].isnull().mean().rename("missing_after")
    miss_report = pd.concat([miss_before, miss_after], axis=1).reset_index()
    miss_report.columns = ["variable", "missing_before", "missing_after"]

    # Also compute per-variable per-hour missingness (pre-imputation) for QC plots
    # We'll recalculate from the raw merge later; here we save the summary
    miss_report.to_csv(os.path.join(OUT_DIR, "missingness_report.csv"), index=False)
    print("    Missingness report saved.")
    return df


def feature_engineering(df):
    print("\n[6.5/7] Feature Engineering (deltas and rolling windows) ...")
    df = df.sort_values(["stay_id", "hour_bin"]).copy()

    # Variables to create temporal features for
    TemporalVars = ["MAP", "HR", "SpO2"]

    new_cols = []
    for var in TemporalVars:
        # 1. Delta (hourly change)
        df[f"{var}_delta"] = df.groupby("stay_id")[var].diff().fillna(0.0)

        # 2. Rolling mean (3h window)
        df[f"{var}_roll3_mean"] = df.groupby("stay_id")[var].transform(
            lambda s: s.rolling(window=3, min_periods=1).mean())

        # 3. Rolling std (3h window) - measures volatility
        df[f"{var}_roll3_std"] = df.groupby("stay_id")[var].transform(
            lambda s: s.rolling(window=3, min_periods=1).std().fillna(0.0))

        new_cols += [f"{var}_delta", f"{var}_roll3_mean", f"{var}_roll3_std"]

```

```python
    print(f"    Added {len(new_cols)} temporal features: {', '.join(new_cols)}")
    return df


def split_dataset(df):
    print("[7/7] Train/val/test split by subject_id (70/15/15, stratified by mortality) ...")
    subject_mort = (df.groupby("subject_id")["hospital_expire_flag"]
                    .max().reset_index()
                    .rename(columns={"hospital_expire_flag": "mortality"}))

    # Stratified split
    train_val, test = train_test_split(
        subject_mort, test_size=0.15, random_state=SEED,
        stratify=subject_mort["mortality"])
    train, val = train_test_split(
        train_val, test_size=0.15 / 0.85, random_state=SEED,
        stratify=train_val["mortality"])

    def label_split(subject_id):
        if subject_id in train["subject_id"].values:
            return "train"
        elif subject_id in val["subject_id"].values:
            return "val"
        return "test"

    df["split"] = df["subject_id"].apply(label_split)

    split_counts = df.groupby("split")["stay_id"].nunique()
    print(f"    Stays per split:\n{split_counts.to_string()}")

    # Verify no patient leakage
    for s1, s2 in [("train", "val"), ("train", "test"), ("val", "test")]:
        s1_subs = set(df[df["split"] == s1]["subject_id"])
        s2_subs = set(df[df["split"] == s2]["subject_id"])
        overlap = s1_subs & s2_subs
        assert len(overlap) == 0, f"Patient leakage between {s1} and {s2}: {overlap}"
    print("    No patient leakage detected OK")

    report = {
        "n_subjects":    {"train": int(train.shape[0]),
                          "val":   int(val.shape[0]),
                          "test":  int(test.shape[0])},
        "n_stays":       split_counts.to_dict(),
        "mortality_rate": df.groupby("split")["hospital_expire_flag"].mean().to_dict(),
        "patient_leakage": {"train_val": 0, "train_test": 0, "val_test": 0},
    }
    with open(os.path.join(OUT_DIR, "split_report.json"), "w") as f:
```

```python
        json.dump(report, f, indent=2)

    return df

def main():
    print("=" * 60)
    print("  MIMIC-IV Continuous Time-Series Dataset Builder")
    print("=" * 60)

    cohort, stats = load_cohort()
    chart_df = load_chart_vitals(cohort)
    lab_df   = load_lab_values(cohort)
    interv_df = load_interventions(cohort)

    df = build_hourly_grid(cohort, chart_df, lab_df, interv_df)

    # Dataset Attrition Part 2: Omitted because of missing MAP in the window
    stays_with_map = df[df["MAP"].notna()]["stay_id"].unique()
    stats["after_map_presence_filter"] = len(stays_with_map)

    # Filter df to only include stays with at least one MAP measurement
    df = df[df["stay_id"].isin(stays_with_map)].copy()

    df = impute(df)
    df = feature_engineering(df)
    df = split_dataset(df)

    # Save filtering statistics
    stats_path = os.path.join(OUT_DIR, "filtering_stats.json")
    with open(stats_path, "w") as f:
        json.dump(stats, f, indent=2)
    print(f"    Filtering statistics saved to {stats_path}")

    # Save full long-format dataset and per-split CSVs
    df.to_csv(os.path.join(OUT_DIR, "long_format.csv"), index=False)
    for split in ["train", "val", "test"]:
        df[df["split"] == split].to_csv(
            os.path.join(OUT_DIR, f"{split}.csv"), index=False)

    print(f"\nOK Saved to {OUT_DIR}")
    print(f"  long_format.csv  : {df.shape}")
    print(f"  train.csv rows   : {(df['split']=='train').sum()}")
    print(f"  val.csv rows     : {(df['split']=='val').sum()}")
    print(f"  test.csv rows    : {(df['split']=='test').sum()}")
    print("=" * 60)


if __name__ == "__main__":
    main()
```

A.2 Step II: Physiological Quality Control

Module 02: Physiological QC and Visualization

```python
"""
02_qc_plots.py
==============
Quality-control visualizations for the processed time-series dataset.
Target: MAP only. Creatinine shown as a covariate for context.

Outputs (saved to outputs/qc/):
  1. sampling_density_heatmap.png  -- observations per variable per hour
  2. missingness_profile.png       -- % missing per variable (before vs after imputation)
  3. map_distribution_by_split.png -- box plots of MAP by train/val/test
  4. patient_trajectories.png      -- 5 representative test patients (MAP + key covariates)
  5. sampling_density_bar.png      -- total observation counts per variable
"""

import json
import os
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np
import pandas as pd
import seaborn as sns

warnings.filterwarnings("ignore")

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(BASE_DIR, "data", "processed")
OUT_DIR  = os.path.join(BASE_DIR, "outputs", "qc")
os.makedirs(OUT_DIR, exist_ok=True)

PALETTE = {"train": "#4C72B0", "val": "#DD8452", "test": "#55A868"}
TARGET_COL     = "MAP"
COVARIATE_COLS = ["HR", "RR", "SpO2", "SysBP", "DiaBP", "Temperature",
                  "Creatinine", "WBC", "Glucose", "Lactate"]
NOTE_COVS      = ["Vasopressor", "IV_Fluid_mL"]   # intervention columns
ALL_VARS       = [TARGET_COL] + COVARIATE_COLS + NOTE_COVS


def load_data():
    df   = pd.read_csv(os.path.join(DATA_DIR, "long_format.csv"))
```

```python
    miss = pd.read_csv(os.path.join(DATA_DIR, "missingness_report.csv"))

    stats_path = os.path.join(DATA_DIR, "filtering_stats.json")
    if os.path.exists(stats_path):
        with open(stats_path, "r") as f:
            stats = json.load(f)
    else:
        stats = None

    import sys
    import importlib.util
    spec = importlib.util.spec_from_file_location("build_dataset", os.path.join(BASE_DIR, "src", "01_build_dataset.py"))
    build_dataset = importlib.util.module_from_spec(spec)
    sys.modules["build_dataset"] = build_dataset
    spec.loader.exec_module(build_dataset)

    # Load raw data pre-imputation for density mapping
    cohort, _ = build_dataset.load_cohort()
    chart_df = build_dataset.load_chart_vitals(cohort)
    lab_df   = build_dataset.load_lab_values(cohort)
    interv_df = build_dataset.load_interventions(cohort)
    df_raw = build_dataset.build_hourly_grid(cohort, chart_df, lab_df, interv_df)

    stays_with_map = df_raw[df_raw["MAP"].notna()]["stay_id"].unique()
    df_raw = df_raw[df_raw["stay_id"].isin(stays_with_map)].copy()

    return df, miss, stats, df_raw

# ---------------------------------------------------------------------------
# Plot 0 -- Dataset Attrition (Filtering)
# ---------------------------------------------------------------------------
def plot_dataset_attrition(stats):
    if not stats:
        print("  Warning: No filtering stats found. Skipping attrition plot.")
        return

    labels = [
        "Raw ICU Stays",
        "Adults (>=18y)",
        "Stay >= 6h",
        "Retained (MAP present)"
    ]
    vals = [
        stats["raw_icustays"],
        stats["after_age_filter"],
        stats["after_los_filter"],
        stats["after_map_presence_filter"]
```

```python
    ]

    # Calculate omitted counts
    omitted = []
    prev = None
    for v in vals:
        if prev is None:
            omitted.append(0)
        else:
            omitted.append(prev - v)
        prev = v

    fig, ax = plt.subplots(figsize=(10, 6))
    x = np.arange(len(labels))
    bars = ax.bar(x, vals, color="#4C72B0", alpha=0.8, width=0.6)

    # Add trend line
    ax.plot(x, vals, marker='o', color='#D65F5F', linewidth=2, markersize=8)

    ax.set_xticks(x)
    ax.set_xticklabels(labels, fontsize=10)
    ax.set_ylabel("Number of ICU Stays", fontsize=11)
    ax.set_title("Dataset Attrition (Inclusion/Exclusion Flow)", fontsize=13, fontweight="bold")

    # Add value labels on top of bars
    for i, (bar, val) in enumerate(zip(bars, vals)):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + (max(vals)*0.02),
                f"{val:,}", ha="center", va="bottom", fontsize=10, fontweight="bold")

        if i > 0 and omitted[i] > 0:
            ax.text(i, vals[i] + (vals[i-1]-vals[i])/2, f"-{omitted[i]:,} omitted",
                    ha="left", va="center", fontsize=9, color="#D65F5F", fontweight="bold",
                    bbox=dict(facecolor='white', alpha=0.7, edgecolor='none'))

    ax.yaxis.grid(True, linestyle="--", alpha=0.6)
    ax.set_axisbelow(True)
    plt.tight_layout()

    path = os.path.join(OUT_DIR, "dataset_attrition.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"  Saved: {path}")



# ----------------------------------------------------------------------------
# Plot 1 -- Sampling density heatmap
```

```python
# ----------------------------------------------------------------------------
def plot_sampling_density_heatmap(df):
    vars_of_interest = [v for v in ALL_VARS if v in df.columns]
    hour_bins = sorted(df["hour_bin"].unique())
    hour_sample = [h for h in hour_bins if h % 2 == 0]

    data_mat = []
    for var in vars_of_interest:
        row = [df[df["hour_bin"] == h][var].notna().mean() for h in hour_sample]
        data_mat.append(row)

    mat = np.array(data_mat)
    fig, ax = plt.subplots(figsize=(16, 5))
    im = ax.imshow(mat, aspect="auto", cmap="YlOrRd", vmin=0, vmax=1)
    ax.set_xticks(range(len(hour_sample)))
    ax.set_xticklabels([str(h) for h in hour_sample], fontsize=7, rotation=90)
    ax.set_yticks(range(len(vars_of_interest)))
    ax.set_yticklabels(vars_of_interest, fontsize=9)
    ax.set_xlabel("Hour relative to ICU admission", fontsize=11)
    ax.set_title("Sampling Density -- Fraction of Patient-Hours with a Recorded Value",
                 fontsize=13, fontweight="bold")
    plt.colorbar(im, ax=ax, label="Fraction present")
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "sampling_density_heatmap.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"  Saved: {path}")


# ----------------------------------------------------------------------------
# Plot 2 -- Missingness profile
# ----------------------------------------------------------------------------
def plot_missingness_profile(miss_df):
    # Highlight MAP vs others
    miss_df = miss_df[miss_df["variable"].isin(ALL_VARS)].copy()
    miss_df = miss_df.sort_values("missing_before", ascending=False)

    fig, ax = plt.subplots(figsize=(13, 5))
    x = np.arange(len(miss_df))
    w = 0.35
    bar_colors_before = ["#D65F5F" if v == TARGET_COL else "#D65F5F"
                         for v in miss_df["variable"]]
    bar_colors_after  = ["#2ca02c" if v == TARGET_COL else "#4C72B0"
```

```
                        for v in miss_df["variable"]]

    bars1 = ax.bar(x - w/2, miss_df["missing_before"] * 100, w,
                   label="Before imputation", color=bar_colors_before, alpha=0.85)
    bars2 = ax.bar(x + w/2, miss_df["missing_after"]  * 100, w,
                   label="After imputation",  color=bar_colors_after,  alpha=0.85)

    ax.set_xticks(x)
    ax.set_xticklabels(miss_df["variable"], rotation=45, ha="right", fontsize=9)
    ax.set_ylabel("% Missing", fontsize=11)
    ax.set_ylim(0, 107)
    ax.set_title("Missingness Profile -- Before vs After Imputation\n"
                 "(green = MAP target after imputation; "
                 "note Creatinine ~94% raw missing -- used as covariate only)",
                 fontsize=11, fontweight="bold")
    ax.legend(fontsize=10)
    ax.yaxis.grid(True, linestyle="--", alpha=0.7)
    ax.set_axisbelow(True)
    for bar in bars1:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2, h + 0.5, f"{h:.0f}%",
                ha="center", va="bottom", fontsize=6.5, color="#8B0000")
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "missingness_profile.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"  Saved: {path}")


# ---------------------------------------------------------------------------
# Plot 3 -- MAP distribution by split
# ---------------------------------------------------------------------------
def plot_map_distribution(df):
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Left: box plot per split
    ax = axes[0]
    data, labels, colors = [], [], []
    for split in ["train", "val", "test"]:
        vals = df[df["split"] == split]["MAP"].dropna()
        data.append(vals.values)
        labels.append(f"{split}\n(n={len(vals):,})")
        colors.append(PALETTE[split])
    bp = ax.boxplot(data, patch_artist=True, notch=True,
                    medianprops=dict(color="white", linewidth=2))
```

```python
    for patch, color in zip(bp["boxes"], colors):
        patch.set_facecolor(color)
        patch.set_alpha(0.8)
    ax.set_xticklabels(labels, fontsize=9)
    ax.set_ylabel("MAP (mmHg)", fontsize=10)
    ax.set_title("MAP Distribution by Split", fontsize=12, fontweight="bold")
    ax.set_ylim(30, 160)
    ax.axhline(65, color="red", linestyle="--", linewidth=1, alpha=0.7,
               label="Shock threshold (65 mmHg)")
    ax.legend(fontsize=8)
    ax.yaxis.grid(True, linestyle="--", alpha=0.6)
    ax.set_axisbelow(True)

    # Right: KDE of MAP per split
    ax2 = axes[1]
    for split, color in PALETTE.items():
        vals = df[df["split"] == split]["MAP"].dropna()
        vals.plot.kde(ax=ax2, label=split, color=color, linewidth=2)
    ax2.axvline(65, color="red", linestyle="--", linewidth=1.5, alpha=0.7)
    ax2.set_xlim(20, 160)
    ax2.set_xlabel("MAP (mmHg)", fontsize=10)
    ax2.set_title("MAP Density by Split", fontsize=12, fontweight="bold")
    ax2.legend(fontsize=9)
    ax2.yaxis.grid(True, linestyle="--", alpha=0.6)
    ax2.set_axisbelow(True)

    plt.suptitle("MAP Target Variable -- Split Consistency Check",
                 fontsize=13, fontweight="bold", y=1.01)
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "map_distribution_by_split.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Saved: {path}")


# ---------------------------------------------------------------------------
# Plot 4 -- Patient trajectories (MAP + HR + Vasopressor)
# ---------------------------------------------------------------------------
def plot_patient_trajectories(df):
    test_df  = df[df["split"] == "test"]
    stay_ids = test_df["stay_id"].unique()
    np.random.seed(42)
    chosen   = np.random.choice(stay_ids, size=min(5, len(stay_ids)), replace=False)

    fig = plt.figure(figsize=(16, 16))
    gs  = gridspec.GridSpec(5, 2, hspace=0.6, wspace=0.32)
```

```python

    for row_i, sid in enumerate(chosen):
        pat = df[df["stay_id"] == sid].sort_values("hour_bin")

        # Left: MAP trajectory
        ax1 = fig.add_subplot(gs[row_i, 0])
        ax1.plot(pat["hour_bin"], pat["MAP"], color="#1f77b4", linewidth=1.8, label="MAP")
        ax1.axhline(65, color="red", linestyle="--", alpha=0.6, linewidth=1,
                    label="65 mmHg threshold")
        ax1.fill_between(pat["hour_bin"], 65, pat["MAP"],
                         where=pat["MAP"] < 65,
                         alpha=0.18, color="red", label="Below threshold")
        ax1.axvline(0, color="gray", linestyle=":", alpha=0.7)
        ax1.set_ylabel("MAP (mmHg)", fontsize=8)
        ax1.set_xlabel("Hour from ICU admission", fontsize=7)
        ax1.set_title(f"stay_id={sid} -- MAP", fontsize=9, fontweight="bold")
        ax1.legend(fontsize=7)
        ax1.grid(True, linestyle="--", alpha=0.5)

        # Right: HR and Vasopressor
        ax2 = fig.add_subplot(gs[row_i, 1])
        ax2.plot(pat["hour_bin"], pat["HR"], color="#2ca02c",
                 linewidth=1.5, label="HR (bpm)")
        ax3 = ax2.twinx()
        if "Vasopressor" in pat.columns:
            ax3.fill_between(pat["hour_bin"], 0, pat["Vasopressor"],
                             color="#d62728", alpha=0.35, label="Vasopressor ON")
            ax3.set_ylabel("Vasopressor (binary)", fontsize=7, color="#d62728")
            ax3.set_ylim(-0.1, 1.5)
        ax2.axvline(0, color="gray", linestyle=":", alpha=0.7)
        ax2.set_ylabel("HR (bpm)", fontsize=8)
        ax2.set_xlabel("Hour from ICU admission", fontsize=7)
        ax2.set_title(f"stay_id={sid} -- HR & Vasopressor", fontsize=9, fontweight="bold")
        ax2.legend(loc="upper left", fontsize=7)
        ax2.grid(True, linestyle="--", alpha=0.5)

    plt.suptitle(
        "Representative Patient Trajectories -- Test Set\n"
        "Left: MAP with 65 mmHg shock threshold | "
        "Right: HR + vasopressor use\n"
        "(vertical dashed = ICU admission time)",
        fontsize=11, fontweight="bold"
    )
    path = os.path.join(OUT_DIR, "patient_trajectories.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Saved: {path}")

```

```python

# ----------------------------------------------------------------------
# Plot 5 -- Sampling density bar chart
# ----------------------------------------------------------------------
def plot_sampling_density_bar(df):
    stay_count  = df["stay_id"].nunique()
    hour_count  = df["hour_bin"].nunique()
    total_slots = stay_count * hour_count

    vars_to_plot = [v for v in ALL_VARS if v in df.columns]
    percentages  = {}
    for col in vars_to_plot:
        if col in ("Vasopressor", "IV_Fluid_mL"):
            percentages[col] = (df[col] > 0).sum() / total_slots * 100
        else:
            percentages[col] = df[col].notna().sum() / total_slots * 100

    vars_sorted = sorted(percentages, key=lambda c: percentages[c], reverse=True)
    vals = [percentages[c] for c in vars_sorted]
    colors = []
    for v in vars_sorted:
        if v == TARGET_COL:
            colors.append("#1f77b4")        # blue = target
        elif v in NOTE_COVS:
            colors.append("#d62728")        # red  = intervention
        elif v == "Creatinine":
            colors.append("#ff7f0e")        # orange = sparse covariate
        else:
            colors.append("#78c679")        # green = regular covariate

    fig, ax = plt.subplots(figsize=(14, 5))
    bars = ax.bar(vars_sorted, vals, color=colors, alpha=0.85, edgecolor="white")
    ax.set_ylabel("% of patient-hours with observation", fontsize=11)
    ax.set_title(
        "Sampling Density per Variable\n"
        "(blue=MAP target | orange=Creatinine covariate [sparse, not a target] | "
        "green=covariates | red=interventions)",
        fontsize=11, fontweight="bold"
    )
    ax.set_ylim(0, 110)
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
```

```
                f"{val:.0f}%", ha="center", va="bottom", fontsize=8)
    plt.xticks(rotation=40, ha="right", fontsize=9)
    ax.yaxis.grid(True, linestyle="--", alpha=0.6)
    ax.set_axisbelow(True)
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "sampling_density_bar.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"  Saved: {path}")


def main():
    print("=" * 60)
    print("  QC Plots -- MAP Target")
    print("=" * 60)
    df, miss_df, stats, df_raw = load_data()
    print(f"Loaded Imputed: {df.shape}  stays={df['stay_id'].nunique()}")
    print(f"Loaded Raw:     {df_raw.shape}")

    print("\n[0/5] Dataset attrition (filtering) ...")
    plot_dataset_attrition(stats)
    print("[1/5] Sampling density heatmap (Using Raw Data) ...")
    plot_sampling_density_heatmap(df_raw)
    print("[2/5] Missingness profile ...")
    plot_missingness_profile(miss_df)
    print("[3/5] MAP distribution by split ...")
    plot_map_distribution(df)
    print("[4/5] Patient trajectories ...")
    plot_patient_trajectories(df)
    print("[5/5] Sampling density bar chart (Using Raw Data) ...")
    plot_sampling_density_bar(df_raw)
    print("\nAll QC plots saved to outputs/qc/")


if __name__ == "__main__":
    main()
```

A.3 Step III: Autoregressive Baseline (Prophet)

Module 03: Univariate Baseline (Facebook Prophet)

```
"""
03_model_prophet.py
===================
Time-Series Model: Facebook Prophet
  - Target:    MAP (Mean Arterial Pressure)
  - Strategy:  Univariate forecasting using trend and seasonality
  - Outputs:   per-patient predictions, aggregate metrics, forecast plots
"""

import os
import warnings
```

```python
from collections import defaultdict
import logging

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from prophet import Prophet

# Suppress Prophet logs
logging.getLogger('prophet').setLevel(logging.ERROR)
logging.getLogger('cmdstanpy').setLevel(logging.ERROR)
warnings.filterwarnings("ignore")

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(
    __file__)))
DATA_DIR = os.path.join(BASE_DIR, "data", "processed")
OUT_DIR  = os.path.join(BASE_DIR, "outputs", "prophet")
os.makedirs(OUT_DIR, exist_ok=True)

HORIZON_H  = 6       # match deep learning pipeline
HISTORY_H  = 24      # minimum history required
TARGET     = "MAP"

def compute_metrics(y_true, y_pred):
    y_true = np.array(y_true, dtype=float)
    y_pred = np.array(y_pred, dtype=float)
    mask   = ~np.isnan(y_true) & ~np.isnan(y_pred)
    yt, yp = y_true[mask], y_pred[mask]
    if len(yt) == 0:
        return {"MAE": np.nan, "RMSE": np.nan, "R2": np.nan, "MAPE
            ": np.nan}
    mae    = np.mean(np.abs(yt - yp))
    rmse   = np.sqrt(np.mean((yt - yp) ** 2))
    ss_res = np.sum((yt - yp) ** 2)
    ss_tot = np.sum((yt - yt.mean()) ** 2)
    r2     = 1 - ss_res / ss_tot if ss_tot > 0 else np.nan
    with np.errstate(divide="ignore", invalid="ignore"):
        mape = np.mean(np.abs((yt - yp) / np.abs(yt))[np.abs(yt) >
            0]) * 100
    return {"MAE": mae, "RMSE": rmse, "R2": r2, "MAPE": mape}

def main():
    print("=" * 60)
    print("  Model: Prophet -- target=MAP")
    print("=" * 60)

    df = pd.read_csv(os.path.join(DATA_DIR, "long_format.csv"))
    test_df  = df[df["split"] == "test"].copy()
    stay_ids = sorted(test_df["stay_id"].unique())
    print(f"Test stays: {len(stay_ids)}")

    all_results  = []
    plot_records = []

```

```python
    all_y_true = []
    all_y_pred = []

    # Storage for evaluation script compatibility (npy files)
    all_preds_npy = []
    all_true_npy  = []

    for i, sid in enumerate(stay_ids):
        pat = test_df[test_df["stay_id"] == sid].sort_values("hour_bin")
        if len(pat) < HISTORY_H + HORIZON_H:
            continue

        history = pat.iloc[:-HORIZON_H].copy()
        future_true = pat.iloc[-HORIZON_H:].copy()

        # Prophet expects 'ds' (datestamp) and 'y' (value)
        # Create dummy timestamps based on hour_bin
        base_time = pd.Timestamp("2026-01-01")
        m_df = history[["hour_bin", TARGET]].rename(columns={"hour_bin": "ds", TARGET: "y"})
        m_df["ds"] = m_df["ds"].apply(lambda x: base_time + pd.Timedelta(hours=x))

        # Fit Prophet
        m = Prophet(
            changepoint_prior_scale=0.05,
            seasonality_prior_scale=10.0,
            yearly_seasonality=False,
            weekly_seasonality=False,
            daily_seasonality=False
        )
        m.add_country_holidays(country_name='US') # Optional, but shouldn't hurt
        m.fit(m_df)

        # Predict
        future = m.make_future_dataframe(periods=HORIZON_H, freq='h')
        forecast = m.predict(future)

        y_pred = forecast["yhat"].values[-HORIZON_H:]
        y_true = future_true[TARGET].values

        # Clip to reasonable MAP range
        y_pred = np.clip(y_pred, 20, 180)

        res_m = compute_metrics(y_true, y_pred)
        all_results.append({"stay_id": sid, **res_m})

        all_y_true.extend(y_true.tolist())
        all_y_pred.extend(y_pred.tolist())

        all_preds_npy.append(y_pred)
        all_true_npy.append(y_true)

```

```
        if len(plot_records) < 5:
            plot_records.append({
                "stay_id":       sid,
                "history_hours": history["hour_bin"].values,
                "history_vals":  history[TARGET].values,
                "hours_true":    future_true["hour_bin"].values,
                "y_true":        y_true,
                "y_pred":        y_pred
            })

    # Combined metrics
    pooled = compute_metrics(np.array(all_y_true), np.array(all_y_pred))
    n_pts  = len(all_y_true)

    res_df = pd.DataFrame(all_results)
    res_df.to_csv(os.path.join(OUT_DIR, "predictions_per_patient.csv"), index=False)

    metric_rows = []
    for metric in ["MAE", "RMSE", "R2", "MAPE"]:
        metric_rows.append({
            "model": "Prophet", "target": TARGET, "metric": metric,
            "value": pooled[metric],
            "std":   res_df[metric].std(),
            "n_patients": len(res_df),
            "n_points": n_pts
        })
    metrics_df = pd.DataFrame(metric_rows)
    metrics_df.to_csv(os.path.join(OUT_DIR, "metrics.csv"), index=False)

    # Save npy files for evaluation script compatibility
    np.save(os.path.join(OUT_DIR, "preds_test.npy"), np.array(all_preds_npy))
    np.save(os.path.join(OUT_DIR, "y_true_test.npy"), np.array(all_true_npy))

    print("\n-- Prophet Metrics (MAP, pooled) --")
    for _, r in metrics_df.iterrows():
        print(f"  {r['metric']:6s}: {r['value']:.4f} (n={len(res_df)})")

    # -- Plots ------------------------------------------------------------

    n_rows = len(plot_records)
    if n_rows > 0:
        fig, axes = plt.subplots(n_rows, 1, figsize=(14, 4 * n_rows))
        if n_rows == 1: axes = [axes]
        for ax, rec in zip(axes, plot_records):
            ax.plot(rec["history_hours"], rec["history_vals"], color="#1f77b4", label="History")
```

```
            ax.plot(rec["hours_true"], rec["y_true"], color="black", linestyle="--", label="Observed")
            ax.plot(rec["hours_true"], rec["y_pred"], color="purple", label="Prophet Forecast")
            ax.axvline(rec["history_hours"][-1], color="gray", linestyle=":")
            ax.set_title(f"stay {rec['stay_id']}")
            ax.legend(fontsize=7)
        plt.tight_layout()
        plt.savefig(os.path.join(OUT_DIR, "forecast_trajectories.png"), dpi=150)
        plt.close()

if __name__ == "__main__":
    main()
```

A.4 Step IV: Attention-based LSTM Recurrent Network

Module 04: Stacked LSTM with Dot-Product Attention

```
"""
04_model_lstm.py
================
Custom Model 1: Multi-variate multi-step LSTM (PyTorch)
  - Target:     MAP (single output)
  - Covariates: HR, RR, SpO2, SysBP, DiaBP, Temperature,
                Creatinine, WBC, Glucose, Lactate, Vasopressor, IV_Fluid_mL
  - Sliding window: 24h lookback -> 12h forecast
  - Architecture: 2-layer stacked LSTM + linear head
  - Training: Adam, MSE, early stopping on validation loss
"""

import os
import time
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

warnings.filterwarnings("ignore")

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(BASE_DIR, "data", "processed")
OUT_DIR  = os.path.join(BASE_DIR, "outputs", "lstm")
os.makedirs(OUT_DIR, exist_ok=True)

```

```python
# -- Hyper-parameters ----------------------------------------------------------
SEQ_LEN     = 24
HORIZON     = 6
BATCH_SIZE  = 32
MAX_EPOCHS  = 150
LR          = 1e-3
PATIENCE    = 20
HIDDEN_DIM  = 128
N_LAYERS    = 2
DROPOUT     = 0.2

TARGET   = "MAP"
FEATURES = ["MAP", "HR", "RR", "SpO2", "SysBP", "DiaBP", "Temperature",
            "Creatinine", "WBC", "Glucose", "Lactate", "Vasopressor", "IV_Fluid_mL",
            "MAP_delta", "MAP_roll3_mean", "MAP_roll3_std",
            "HR_delta", "HR_roll3_mean", "HR_roll3_std",
            "SpO2_delta", "SpO2_roll3_mean", "SpO2_roll3_std"]
N_FEAT   = len(FEATURES)
DEVICE   = torch.device("cuda" if torch.cuda.is_available() else "cpu")
TGT_IDX  = FEATURES.index(TARGET)   # index 0


# ----------------------------------------------------------------------------
# Dataset construction
# ----------------------------------------------------------------------------
def make_windows(df, split_name):
    sub = df[df["split"] == split_name].sort_values(["stay_id", "hour_bin"])
    for col in FEATURES:
        if col not in sub.columns:
            sub[col] = 0.0

    Xs, ys = [], []
    for sid, pat in sub.groupby("stay_id"):
        vals = pat[FEATURES].values.astype(np.float32)  # (T, F)
        if len(vals) < SEQ_LEN + HORIZON:
            continue
        for start in range(len(vals) - SEQ_LEN - HORIZON + 1):
            Xs.append(vals[start : start + SEQ_LEN])
            # only MAP for target
            ys.append(vals[start + SEQ_LEN : start + SEQ_LEN + HORIZON, TGT_IDX])

    if not Xs:
        return None, None
    return np.array(Xs, dtype=np.float32), np.array(ys, dtype=np.float32)
    # shapes: (N, SEQ_LEN, N_FEAT), (N, HORIZON)
```

```python


def normalise(X_tr, X_vl, X_te):
    mean = X_tr.mean(axis=(0, 1), keepdims=True)
    std  = X_tr.std(axis=(0, 1),  keepdims=True) + 1e-8
    return (X_tr-mean)/std, (X_vl-mean)/std, (X_te-mean)/std, mean, std


def normalise_targets(y_tr, y_vl, y_te):
    mean = float(y_tr.mean())
    std  = float(y_tr.std()) + 1e-8
    return (y_tr-mean)/std, (y_vl-mean)/std, (y_te-mean)/std, mean, std


# ----------------------------------------------------------------------
# Model
# ----------------------------------------------------------------------
class MultiStepLSTM(nn.Module):
    def __init__(self, n_feat, hidden, n_layers, dropout, horizon):
        super().__init__()
        self.lstm = nn.LSTM(n_feat, hidden, n_layers,
                            batch_first=True, dropout=dropout)

        # Dot-product attention
        self.query = nn.Linear(hidden, hidden)
        self.key   = nn.Linear(hidden, hidden)
        self.value = nn.Linear(hidden, hidden)

        self.head = nn.Sequential(
            nn.Linear(hidden, 64),
            nn.ReLU(),
            nn.Linear(64, horizon),
        )
        self.horizon = horizon

    def forward(self, x):
        # x: (B, seq_len, feat)
        lstm_out, (h_n, c_n) = self.lstm(x) # lstm_out: (B, seq_len, hidden)

        # Last hidden state as query
        q = self.query(h_n[-1]).unsqueeze(1) # (B, 1, hidden)
        k = self.key(lstm_out)               # (B, seq_len, hidden)
        v = self.value(lstm_out)             # (B, seq_len, hidden)

        # Attention scores
```

```python
        attn_weights = torch.bmm(q, k.transpose(1, 2)) # (B, 1, seq_len)
        attn_weights = torch.softmax(attn_weights, dim=-1)

        # Context vector
        context = torch.bmm(attn_weights, v).squeeze(1) # (B, hidden)

        # Residual connection
        context = context + h_n[-1]

        return self.head(context)    # (B, horizon)


# ---------------------------------------------------------------------------
# Training
# ---------------------------------------------------------------------------
def train_model(model, tr_dl, vl_dl):
    opt    = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=1e-5)
    crit   = nn.MSELoss()
    sched  = torch.optim.lr_scheduler.ReduceLROnPlateau(
        opt, mode="min", factor=0.5, patience=5, min_lr=1e-5)
    best_v = np.inf
    best_s = None
    p_cnt  = 0
    tr_losses, vl_losses = [], []

    for epoch in range(1, MAX_EPOCHS + 1):
        model.train()
        tl = 0.0
        for xb, yb in tr_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            loss = crit(model(xb), yb)
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            tl += loss.item() * len(xb)
        tl /= len(tr_dl.dataset)
        tr_losses.append(tl)

        model.eval()
        vl = 0.0
        with torch.no_grad():
            for xb, yb in vl_dl:
                vl += crit(model(xb.to(DEVICE)), yb.to(DEVICE)).item() * len(xb)
        vl /= len(vl_dl.dataset)
        vl_losses.append(vl)
        sched.step(vl)

```

```python
        if vl < best_v - 1e-5:
            best_v = vl
            best_s = {k: v.clone() for k, v in model.state_dict().items()}
            p_cnt  = 0
        else:
            p_cnt += 1

        if epoch % 10 == 0:
            current_lr = opt.param_groups[0]["lr"]
            print(f"    Epoch {epoch:3d}: train={tl:.4f}  val={vl:.4f}  lr={current_lr:.2e}")
        if p_cnt >= PATIENCE:
            print(f"    Early stopping at epoch {epoch}")
            break

    if best_s:
        model.load_state_dict(best_s)
    return tr_losses, vl_losses


# ---------------------------------------------------------------------------
# Metrics
# ---------------------------------------------------------------------------
def compute_metrics(y_true, y_pred):
    """y_true, y_pred: (N, HORIZON) in original scale."""
    yt = y_true.flatten()
    yp = y_pred.flatten()
    mask = ~np.isnan(yt) & ~np.isnan(yp)
    yt, yp = yt[mask], yp[mask]
    mae  = np.mean(np.abs(yt - yp))
    rmse = np.sqrt(np.mean((yt - yp) ** 2))
    ss_r = np.sum((yt - yp) ** 2)
    ss_t = np.sum((yt - yt.mean()) ** 2)
    r2   = 1 - ss_r / ss_t if ss_t > 0 else np.nan
    with np.errstate(divide="ignore", invalid="ignore"):
        mape = np.mean(np.abs((yt - yp) / np.abs(yt))[np.abs(yt) > 0]) * 100
    return {"MAE": mae, "RMSE": rmse, "R2": r2, "MAPE": mape}


# ---------------------------------------------------------------------------
# Plots
# ---------------------------------------------------------------------------
def plot_forecasts(X_te, y_true, y_pred, df_test, n=5):
    sub = df_test.sort_values(["stay_id", "hour_bin"])
    stay_ids = sub["stay_id"].unique()
    stay_map = {}
```

```
    w = 0
    for sid in stay_ids:
        pat = sub[sub["stay_id"] == sid]
        nw  = max(0, len(pat) - SEQ_LEN - HORIZON + 1)
        if nw > 0:
            stay_map[sid] = w
            w += nw

    chosen = list(stay_map.keys())[:n]
    if not chosen:
        return

    fig, axes = plt.subplots(len(chosen), 1, figsize=(14, 4 * len(chosen)))
    if len(chosen) == 1:
        axes = [axes]

    for ax, sid in zip(axes, chosen):
        wi  = stay_map[sid]
        hist_h = np.arange(-SEQ_LEN + 1, 1)
        fore_h = np.arange(1, HORIZON + 1)
        hist_v = X_te[wi, :, TGT_IDX]

        ax.plot(hist_h, hist_v, color="#1f77b4", linewidth=1.5, label="History")
        ax.plot(fore_h, y_true[wi], color="black", linewidth=2,
                linestyle="--", label="Observed")
        ax.plot(fore_h, y_pred[wi], color="orange", linewidth=2,
                marker="o", markersize=4, label="LSTM forecast")
        ax.axvline(0, color="gray", linestyle=":", linewidth=1.5, alpha=0.7)
        ax.axhline(65, color="red", linestyle=":", linewidth=1, alpha=0.7)
        ax.set_title(f"stay {sid} -- MAP", fontsize=10, fontweight="bold")
        ax.set_xlabel("Hour relative to forecast origin", fontsize=8)
        ax.set_ylabel("MAP (mmHg)", fontsize=8)
        ax.legend(fontsize=7, ncol=4)
        ax.grid(True, linestyle="--", alpha=0.5)

    plt.suptitle("LSTM -- MAP Forecasts on Test Patients",
                 fontsize=13, fontweight="bold")
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "forecast_trajectories.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Forecast plot saved: {path}")


def plot_training(tr_losses, vl_losses):
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(tr_losses, label="Train", color="#1f77b4")
    ax.plot(vl_losses, label="Val",   color="#d62728")
    ax.set_xlabel("Epoch"); ax.set_ylabel("MSE (normalised)")
```

```python
    ax.set_title("LSTM Training & Validation Loss -- MAP", fontweight="bold")
    ax.legend(); ax.grid(True, linestyle="--", alpha=0.5)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "training_curves.png"), dpi=150)
    plt.close()


def plot_horizon_mae(y_true, y_pred):
    """MAE per forecast horizon step."""
    mae_per_step = np.mean(np.abs(y_true - y_pred), axis=0)   # (HORIZON,)
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.bar(np.arange(1, HORIZON + 1), mae_per_step, color="#1f77b4", alpha=0.85)
    ax.set_xlabel("Forecast horizon (h)", fontsize=10)
    ax.set_ylabel("MAE (mmHg)", fontsize=10)
    ax.set_title("LSTM -- MAP Forecast Error by Horizon Step", fontweight="bold")
    ax.yaxis.grid(True, linestyle="--", alpha=0.6)
    ax.set_axisbelow(True)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "horizon_mae.png"), dpi=150)
    plt.close()
    print(f"  Horizon MAE plot saved.")


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    print("=" * 60)
    print(f"  Model: LSTM -- target=MAP  (device={DEVICE})")
    print("=" * 60)
    t0 = time.time()

    df = pd.read_csv(os.path.join(DATA_DIR, "long_format.csv"))
    df_test = df[df["split"] == "test"].copy()

    print("Building sliding-window datasets ...")
    X_tr, y_tr = make_windows(df, "train")
    X_vl, y_vl = make_windows(df, "val")
    X_te, y_te = make_windows(df, "test")

    if X_tr is None or X_te is None:
        print("ERROR: not enough data."); return

    print(f"  Windows  train={len(X_tr)}  val={len(X_vl)}  test={len(X_te)}")

```

```python
    X_tr_n, X_vl_n, X_te_n, f_mean, f_std = normalise(X_tr, X_vl, X_te)
    y_tr_n, y_vl_n, y_te_n, t_mean, t_std = normalise_targets(y_tr, y_vl, y_te)
    np.save(os.path.join(OUT_DIR, "feat_mean.npy"), f_mean)
    np.save(os.path.join(OUT_DIR, "feat_std.npy"),  f_std)

    def dl(X, y, shuf=False):
        ds = TensorDataset(torch.tensor(X), torch.tensor(y))
        return DataLoader(ds, batch_size=BATCH_SIZE, shuffle=shuf, num_workers=0)

    tr_dl = dl(X_tr_n, y_tr_n, shuf=True)
    vl_dl = dl(X_vl_n, y_vl_n)
    te_dl = dl(X_te_n, y_te_n)

    model = MultiStepLSTM(N_FEAT, HIDDEN_DIM, N_LAYERS, DROPOUT, HORIZON).to(DEVICE)
    print(f"  LSTM parameters: {sum(p.numel() for p in model.parameters()):,}")

    print("\nTraining ...")
    tr_l, vl_l = train_model(model, tr_dl, vl_dl)
    plot_training(tr_l, vl_l)
    torch.save(model.state_dict(), os.path.join(OUT_DIR, "lstm_weights.pt"))

    model.eval()
    preds_n = []
    with torch.no_grad():
        for xb, _ in te_dl:
            preds_n.append(model(xb.to(DEVICE)).cpu().numpy())
    preds_n = np.concatenate(preds_n, 0)  # (N, HORIZON)

    y_pred_raw = preds_n * t_std + t_mean
    y_true_raw = y_te_n * t_std + t_mean

    np.save(os.path.join(OUT_DIR, "preds_test.npy"),  y_pred_raw)
    np.save(os.path.join(OUT_DIR, "y_true_test.npy"), y_true_raw)
    np.save(os.path.join(OUT_DIR, "X_test_raw.npy"),  X_te)

    metrics = compute_metrics(y_true_raw, y_pred_raw)
    metric_rows = [{"model": "LSTM", "target": TARGET,
                    "metric": k, "value": v} for k, v in metrics.items()]
    pd.DataFrame(metric_rows).to_csv(os.path.join(OUT_DIR, "metrics.csv"), index=False)

    print("\n-- LSTM Metrics (MAP, test set) --")
    for k, v in metrics.items():
        print(f"  {k:6s}: {v:.4f}")

    plot_forecasts(X_te, y_true_raw, y_pred_raw, df_test)
    plot_horizon_mae(y_true_raw, y_pred_raw)

    print(f"\nLSTM complete ({time.time()-t0:.1f}s).")
```

```python


if __name__ == "__main__":
    main()
```

A.5 Step V: Temporal Fusion Transformer / Gated Recurrent Units

Module 05: Temporal Fusion Transformer and Fallback GRU

```python
"""
05_model_tft.py
===============
Custom Model 2: Temporal Fusion Transformer / GRU fallback
  - Target:     MAP (single output)
  - Covariates: HR, RR, SpO2, SysBP, DiaBP, Temperature,
                Creatinine, WBC, Glucose, Lactate, IV_Fluid_mL
  - Categorical: Vasopressor (binary)
  - Encoder: 24h -> Prediction: 12h
"""

import os
import time
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

warnings.filterwarnings("ignore")

try:
    import pytorch_lightning as pl
    from pytorch_forecasting import TimeSeriesDataSet,
        TemporalFusionTransformer
    from pytorch_forecasting.data import GroupNormalizer
    from pytorch_forecasting.metrics import MAE as PFMAE
    TFT_AVAILABLE = True
except ImportError:
    TFT_AVAILABLE = False
    print("WARNING: pytorch_forecasting not available -- using GRU
        fallback.")

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(
    __file__)))
DATA_DIR = os.path.join(BASE_DIR, "data", "processed")
OUT_DIR  = os.path.join(BASE_DIR, "outputs", "tft")
os.makedirs(OUT_DIR, exist_ok=True)

ENCODER_LEN = 24
PREDICT_LEN = 6
TARGET      = "MAP"
```

```python
REAL_COVS   = ["HR", "RR", "SpO2", "SysBP", "DiaBP", "Temperature",
               "Creatinine", "WBC", "Glucose", "Lactate", "IV_Fluid_mL",
               "MAP_delta", "MAP_roll3_mean", "MAP_roll3_std",
               "HR_delta", "HR_roll3_mean", "HR_roll3_std",
               "SpO2_delta", "SpO2_roll3_mean", "SpO2_roll3_std"]
FEATURES    = [TARGET] + REAL_COVS + ["Vasopressor"]
N_FEAT      = len(FEATURES)
TGT_IDX     = 0
DEVICE      = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# ---------------------------------------------------------------------
# Metrics
# ---------------------------------------------------------------------
def compute_metrics(y_true, y_pred):
    yt = np.array(y_true, dtype=float).flatten()
    yp = np.array(y_pred, dtype=float).flatten()
    mask = ~np.isnan(yt) & ~np.isnan(yp)
    yt, yp = yt[mask], yp[mask]
    if len(yt) == 0:
        return {"MAE": np.nan, "RMSE": np.nan, "R2": np.nan, "MAPE": np.nan}
    mae  = np.mean(np.abs(yt - yp))
    rmse = np.sqrt(np.mean((yt - yp) ** 2))
    ss_r = np.sum((yt - yp) ** 2)
    ss_t = np.sum((yt - yt.mean()) ** 2)
    r2   = 1 - ss_r / ss_t if ss_t > 0 else np.nan
    with np.errstate(divide="ignore", invalid="ignore"):
        mape = np.mean(np.abs((yt - yp) / np.abs(yt))[np.abs(yt) > 0]) * 100
    return {"MAE": mae, "RMSE": rmse, "R2": r2, "MAPE": mape}


# ---------------------------------------------------------------------
# Shared forecast plots
# ---------------------------------------------------------------------
def plot_forecasts(X_te, y_true, y_pred, df_test, label="TFT/GRU", n=5):
    sub = df_test.sort_values(["stay_id", "hour_bin"])
    stay_ids = sub["stay_id"].unique()
    stay_map = {}
    w = 0
    for sid in stay_ids:
        nw = max(0, len(sub[sub["stay_id"] == sid]) - ENCODER_LEN - PREDICT_LEN + 1)
```

```
        if nw > 0:
            stay_map[sid] = w
            w += nw

    chosen = list(stay_map.keys())[:n]
    if not chosen:
        return

    fig, axes = plt.subplots(len(chosen), 1, figsize=(14, 4 * len(
        chosen)))
    if len(chosen) == 1:
        axes = [axes]

    for ax, sid in zip(axes, chosen):
        wi = stay_map[sid]
        hist_h = np.arange(-ENCODER_LEN + 1, 1)
        fore_h = np.arange(1, PREDICT_LEN + 1)
        hist_v = X_te[wi, :, TGT_IDX] if X_te is not None else np.
            full(ENCODER_LEN, np.nan)

        ax.plot(hist_h, hist_v, color="#1f77b4", linewidth=1.5,
            label="History")
        ax.plot(fore_h, y_true[wi], color="black", linewidth=2,
                linestyle="--", label="Observed")
        ax.plot(fore_h, y_pred[wi], color="#2ca02c", linewidth=2,
                marker="s", markersize=4, label=f"{label} forecast
                    ")
        ax.axhline(65, color="red", linestyle=":", linewidth=1,
            alpha=0.7)
        ax.axvline(0, color="gray", linestyle=":", linewidth=1.5,
            alpha=0.7)
        ax.set_title(f"stay {sid} -- MAP", fontsize=10, fontweight
            ="bold")
        ax.set_xlabel("Hour relative to forecast origin", fontsize
            =8)
        ax.set_ylabel("MAP (mmHg)", fontsize=8)
        ax.legend(fontsize=7, ncol=4)
        ax.grid(True, linestyle="--", alpha=0.5)

    plt.suptitle(f"{label} -- MAP Forecasts on Test Patients",
                 fontsize=13, fontweight="bold")
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "forecast_trajectories.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Forecast plot saved: {path}")


def plot_horizon_mae(y_true, y_pred):
    mae_per_step = np.mean(np.abs(y_true - y_pred), axis=0)
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.bar(np.arange(1, PREDICT_LEN + 1), mae_per_step,
           color="#2ca02c", alpha=0.85)
    ax.set_xlabel("Forecast horizon (h)", fontsize=10)
    ax.set_ylabel("MAE (mmHg)", fontsize=10)
```

```python
    ax.set_title("GRU/TFT -- MAP Forecast Error by Horizon Step", fontweight="bold")
    ax.yaxis.grid(True, linestyle="--", alpha=0.6)
    ax.set_axisbelow(True)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "horizon_mae.png"), dpi=150)
    plt.close()


# ---------------------------------------------------------------------------
# GRU fallback
# ---------------------------------------------------------------------------
class GRUModel(torch.nn.Module):
    def __init__(self, n_feat, hidden=128, n_layers=2, dropout=0.2, horizon=12):
        super().__init__()
        self.gru  = torch.nn.GRU(n_feat, hidden, n_layers,
                                 batch_first=True, dropout=dropout)
        self.head = torch.nn.Sequential(
            torch.nn.Linear(hidden, 64),
            torch.nn.ReLU(),
            torch.nn.Linear(64, horizon),
        )

    def forward(self, x):
        out, _ = self.gru(x)
        return self.head(out[:, -1])    # (B, horizon)


def run_gru_fallback(df):
    from torch.utils.data import TensorDataset, DataLoader

    for col in FEATURES:
        if col not in df.columns:
            df[col] = 0.0

    def make_windows(split):
        sub = df[df["split"] == split].sort_values(["stay_id", "hour_bin"])
        Xs, ys = [], []
        for _, pat in sub.groupby("stay_id"):
            vals = pat[FEATURES].values.astype(np.float32)
            if len(vals) < ENCODER_LEN + PREDICT_LEN:
                continue
            for start in range(len(vals) - ENCODER_LEN - PREDICT_LEN + 1):
                Xs.append(vals[start : start + ENCODER_LEN])
                ys.append(vals[start + ENCODER_LEN : start + ENCODER_LEN + PREDICT_LEN, TGT_IDX])
        return (np.array(Xs, np.float32), np.array(ys, np.float32)) if Xs else (None, None)
```

```python

    X_tr, y_tr = make_windows("train")
    X_vl, y_vl = make_windows("val")
    X_te, y_te = make_windows("test")
    if X_tr is None or X_te is None:
        return None, None, None

    f_mean = X_tr.mean((0, 1), keepdims=True)
    f_std  = X_tr.std((0, 1),  keepdims=True) + 1e-8
    X_tr_n, X_vl_n, X_te_n = (X_tr-f_mean)/f_std, (X_vl-f_mean)/f_std, (X_te-f_mean)/f_std
    t_mean = float(y_tr.mean())
    t_std  = float(y_tr.std()) + 1e-8
    y_tr_n = (y_tr - t_mean) / t_std
    y_vl_n = (y_vl - t_mean) / t_std
    y_te_n = (y_te - t_mean) / t_std

    model = GRUModel(N_FEAT, horizon=PREDICT_LEN).to(DEVICE)
    opt   = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
    crit  = torch.nn.MSELoss()
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(
        opt, mode="min", factor=0.5, patience=5, min_lr=1e-5)

    tr_ds = DataLoader(TensorDataset(torch.tensor(X_tr_n), torch.tensor(y_tr_n)),
                       batch_size=32, shuffle=True)
    vl_ds = DataLoader(TensorDataset(torch.tensor(X_vl_n), torch.tensor(y_vl_n)),
                       batch_size=64)
    te_ds = DataLoader(TensorDataset(torch.tensor(X_te_n), torch.tensor(y_te_n)),
                       batch_size=64)

    best_v, best_s, pat_cnt = np.inf, None, 0
    tr_losses, vl_losses = [], []
    MAX_EPOCHS_GRU = 150
    PATIENCE_GRU   = 20

    for epoch in range(1, MAX_EPOCHS_GRU + 1):
        # ---- proper training step ----
        model.train()
        tl = 0.0
        for xb, yb in tr_ds:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            loss = crit(model(xb), yb)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            tl += loss.item() * len(xb)
        tl /= len(tr_ds.dataset)

        # ---- validation ----
        model.eval()
```

```
        vl = 0.0
        with torch.no_grad():
            for xb, yb in vl_ds:
                vl += crit(model(xb.to(DEVICE)), yb.to(DEVICE)).
                    item() * len(xb)
        vl /= len(vl_ds.dataset)

        tr_losses.append(tl)
        vl_losses.append(vl)
        sched.step(vl)

        if vl < best_v - 1e-5:
            best_v  = vl
            best_s  = {k: v.clone() for k, v in model.state_dict()
                .items()}
            pat_cnt = 0
        else:
            pat_cnt += 1

        if epoch % 10 == 0:
            current_lr = opt.param_groups[0]["lr"]
            print(f"    Epoch {epoch:3d}: train={tl:.4f}  val={vl
                :.4f}  lr={current_lr:.2e}")
        if pat_cnt >= PATIENCE_GRU:
            print(f"    Early stopping at epoch {epoch}")
            break

    if best_s:
        model.load_state_dict(best_s)
    model.eval()
    preds_n = []
    with torch.no_grad():
        for xb, _ in te_ds:
            preds_n.append(model(xb.to(DEVICE)).cpu().numpy())
    preds_n = np.concatenate(preds_n, 0)
    y_pred_raw = preds_n * t_std + t_mean
    y_true_raw = y_te_n * t_std + t_mean

    # Training curves
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(tr_losses, label="Train", color="#1f77b4")
    ax.plot(vl_losses, label="Val",   color="#d62728")
    ax.set_xlabel("Epoch"); ax.set_ylabel("MSE (normalised)")
    ax.set_title("GRU/TFT Training Curves -- MAP", fontweight="
        bold")
    ax.legend(); ax.grid(True, linestyle="--", alpha=0.5)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "training_curves.png"), dpi
        =150)
    plt.close()

    torch.save(model.state_dict(), os.path.join(OUT_DIR, "
        tft_weights.pt"))
    return X_te, y_true_raw, y_pred_raw


```

```python
# ----------------------------------------------------------------------
# Main
# ----------------------------------------------------------------------
def main():
    print("=" * 60)
    print(f"  Model: TFT/GRU -- target=MAP  (device={DEVICE})")
    if not TFT_AVAILABLE:
        print("  [GRU fallback active]")
    print("=" * 60)
    t0 = time.time()

    df      = pd.read_csv(os.path.join(DATA_DIR, "long_format.csv"))
    df_test = df[df["split"] == "test"].copy()

    if TFT_AVAILABLE:
        try:
            X_te, y_true_raw, y_pred_raw = run_tft(df)
        except Exception as exc:
            print(f"  TFT failed ({exc}); using GRU fallback ...")
            X_te, y_true_raw, y_pred_raw = run_gru_fallback(df)
    else:
        X_te, y_true_raw, y_pred_raw = run_gru_fallback(df)

    if y_true_raw is None:
        print("ERROR: insufficient data."); return

    np.save(os.path.join(OUT_DIR, "preds_test.npy"),  y_pred_raw)
    np.save(os.path.join(OUT_DIR, "y_true_test.npy"), y_true_raw)
    if X_te is not None:
        np.save(os.path.join(OUT_DIR, "X_test_raw.npy"), X_te)

    metrics = compute_metrics(y_true_raw, y_pred_raw)
    rows = [{"model": "TFT/GRU", "target": TARGET,
             "metric": k, "value": v} for k, v in metrics.items()]
    pd.DataFrame(rows).to_csv(os.path.join(OUT_DIR, "metrics.csv"), index=False)

    print("\n-- TFT/GRU Metrics (MAP, test set) --")
    for k, v in metrics.items():
        print(f"  {k:6s}: {v:.4f}")

    plot_forecasts(X_te, y_true_raw, y_pred_raw, df_test)
    plot_horizon_mae(y_true_raw, y_pred_raw)

    print(f"\nTFT/GRU complete ({time.time()-t0:.1f}s).")


if __name__ == "__main__":
    main()
```

A.6 Step VI: Statistical Evaluation and Metric Reporting

Module 06: Comprehensive Metric Aggregation and Reporting

```python
"""
06_evaluate_and_report.py
=========================
Aggregate results from all 3 models (Prophet, LSTM, TFT/GRU).
Target: MAP only.

Outputs -> outputs/results/:
  leaderboard.csv           -- MAE/RMSE/R2/MAPE per model
  leaderboard_plot.png      -- bar chart
  forecast_overlay.png      -- 5 test patients, all 3 models overlaid
  horizon_comparison.png    -- MAE per horizon step (h+1 ... h+12)
  error_distribution.png    -- per-patient MAE box plots
  summary.txt               -- plain-English interpretation
"""

import os
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

BASE_DIR    = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR    = os.path.join(BASE_DIR, "data", "processed")
OUT_PROPHET = os.path.join(BASE_DIR, "outputs", "prophet")
OUT_LSTM    = os.path.join(BASE_DIR, "outputs", "lstm")
OUT_TFT     = os.path.join(BASE_DIR, "outputs", "tft")
OUT_ENS     = os.path.join(BASE_DIR, "outputs", "ensemble")
OUT_DIR     = os.path.join(BASE_DIR, "outputs", "results")
os.makedirs(OUT_DIR, exist_ok=True)

TARGET         = "MAP"
MODEL_DIRS     = {"Prophet": OUT_PROPHET, "LSTM": OUT_LSTM, "TFT/GRU": OUT_TFT, "Ensemble": OUT_ENS}
MODEL_COLORS   = {"Prophet": "#e377c2", "LSTM": "#1f77b4", "TFT/GRU": "#2ca02c", "Ensemble": "#d62728"}
METRICS_ORDER  = ["MAE", "RMSE", "R2", "MAPE"]
ENCODER_LEN    = 24
PREDICT_LEN    = 6


def load_array(path):
    return np.load(path) if os.path.exists(path) else None


```

```python
# ---------------------------------------------------------------------------
# 1. Leaderboard
# ---------------------------------------------------------------------------
def build_leaderboard():
    rows = []
    for model, mdir in MODEL_DIRS.items():
        mp = os.path.join(mdir, "metrics.csv")
        if not os.path.exists(mp):
            print(f"  WARNING: {mp} missing -- skipping {model}")
            continue
        mdf = pd.read_csv(mp)
        if "metric" in mdf.columns and "value" in mdf.columns:
            for _, r in mdf.iterrows():
                rows.append({"Model": model, "Metric": r["metric"],
                             "Value": r["value"]})
        else:
            for metric in METRICS_ORDER:
                if metric in mdf.columns:
                    rows.append({"Model": model, "Metric": metric,
                                 "Value": mdf[metric].mean()})

    if not rows:
        print("ERROR: no metrics found."); return None

    lb = pd.DataFrame(rows)
    lb_pivot = lb.pivot_table(index="Model", columns="Metric",
                              values="Value").reset_index()
    lb_pivot.to_csv(os.path.join(OUT_DIR, "leaderboard.csv"), index=False)
    lb.to_csv(os.path.join(OUT_DIR, "leaderboard_long.csv"), index=False)
    print(f"  Leaderboard saved ({len(lb_pivot)} models).")
    return lb


# ---------------------------------------------------------------------------
# 2. Leaderboard bar chart
# ---------------------------------------------------------------------------
def plot_leaderboard(lb):
    if lb is None or lb.empty: return
    metrics       = ["MAE", "RMSE", "R2", "MAPE"]
    better_lower  = {"MAE": True, "RMSE": True, "R2": False, "MAPE": True}

    fig, axes = plt.subplots(2, 2, figsize=(14, 8))
    for ax, metric in zip(axes.flatten(), metrics):
        sub    = lb[lb["Metric"] == metric].copy()
```

```
        models = [m for m in MODEL_DIRS if m in sub["Model"].values]
        vals   = [sub[sub["Model"] == m]["Value"].values[0] for m in models]
        colors = [MODEL_COLORS.get(m, "#888") for m in models]
        bars   = ax.bar(models, vals, color=colors, alpha=0.85, edgecolor="white")
        for bar, val in zip(bars, vals):
            if not np.isnan(val):
                ax.text(bar.get_x() + bar.get_width()/2,
                        bar.get_height() * (1.01 if val > 0 else 0.99),
                        f"{val:.3f}", ha="center", va="bottom", fontsize=9,
                        fontweight="bold")
        arrow = "(lower is better)" if better_lower[metric] else "(higher is better)"
        ax.set_title(f"{metric}  {arrow}", fontsize=11, fontweight="bold")
        ax.yaxis.grid(True, linestyle="--", alpha=0.6)
        ax.set_axisbelow(True)
        ax.tick_params(axis="x", labelsize=10)

    plt.suptitle(f"Model Performance Leaderboard -- {TARGET} Forecasting",
                 fontsize=14, fontweight="bold")
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "leaderboard_plot.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Leaderboard plot saved: {path}")


# ---------------------------------------------------------------------------
# 3. Forecast overlay (5 patients, all 3 models)
# ---------------------------------------------------------------------------
def plot_forecast_overlay(df_test):
    # Load data
    lstm_true = load_array(os.path.join(OUT_LSTM, "y_true_test.npy"))
    lstm_X    = load_array(os.path.join(OUT_LSTM, "X_test_raw.npy"))

    model_preds = {}
    for name, mdir in MODEL_DIRS.items():
        p = load_array(os.path.join(mdir, "preds_test.npy"))
        if p is not None:
            model_preds[name] = p

    # Mapping
    sub = df_test.sort_values(["stay_id", "hour_bin"])
    stay_map = {}
```

```
    w = 0
    for sid in sub["stay_id"].unique():
        nw = max(0, len(sub[sub["stay_id"] == sid]) - ENCODER_LEN - PREDICT_LEN + 1)
        if nw > 0:
            stay_map[sid] = w
            w += nw

    chosen = list(stay_map.keys())[:5]
    if not chosen: return

    # Individual plots directory
    patient_dir = os.path.join(OUT_DIR, "patients")
    os.makedirs(patient_dir, exist_ok=True)

    # Styling
    model_styles = {
        "Prophet":  dict(linestyle="--", marker="v", alpha=0.6, ms=5),
        "LSTM":     dict(linestyle="-",  marker="o", alpha=0.7, ms=5),
        "TFT/GRU":  dict(linestyle="-.", marker="s", alpha=0.7, ms=5),
        "Ensemble": dict(linestyle="-",  marker="D", alpha=0.9, linewidth=2.5, ms=6),
    }

    def plot_single_stay(ax, sid, wi, is_individual=False):
        hist_h = np.arange(-ENCODER_LEN + 1, 1)
        fore_h = np.arange(1, PREDICT_LEN + 1)

        # History
        if lstm_X is not None and wi < len(lstm_X):
            ax.plot(hist_h, lstm_X[wi, :, 0], color="#555555",
                    linewidth=2, label="History", alpha=0.8)

        # Observed
        if lstm_true is not None and wi < len(lstm_true):
            ax.plot(fore_h, lstm_true[wi], color="black", linewidth=3,
                    linestyle=":", label="Observed", zorder=5)

        # Models
        for model_name, preds_array in model_preds.items():
            if wi < len(preds_array):
                style = model_styles.get(model_name, {})
                color = MODEL_COLORS.get(model_name, "blue")
                ax.plot(fore_h, preds_array[wi], color=color,
                        label=model_name, zorder=4, **style)

        ax.axhline(65, color="#e74c3c", linestyle="--", linewidth=1.2, alpha=0.7, label="MAP < 65")
        ax.axvline(0, color="black", linestyle="-", linewidth=1.5, alpha=0.3)

```

```
        ax.set_title(f"Patient Stay ID: {sid}", fontsize=12, fontweight="bold", pad=10)
        ax.set_xlabel("Hours from Origin (t=0)", fontsize=10)
        ax.set_ylabel("MAP (mmHg)", fontsize=10)
        ax.grid(True, linestyle="--", alpha=0.3)

        if is_individual:
            ax.legend(fontsize=9, loc="upper left", bbox_to_anchor=(1, 1))
        else:
            ax.legend(fontsize=8, ncol=6, loc="upper center", bbox_to_anchor=(0.5, -0.15))

    # 1. Combined Aggregate Plot
    fig, axes = plt.subplots(len(chosen), 1, figsize=(14, 4.5 * len(chosen)))
    if len(chosen) == 1: axes = [axes]

    for ax, sid in zip(axes, chosen):
        plot_single_stay(ax, sid, stay_map[sid])

    plt.suptitle(f"Time-Series Forecasting Overlay (Target: {TARGET})\n"
                 "Phase 4: Multi-Model Precision Comparison (6h Horizon)",
                 fontsize=16, fontweight="bold", y=0.98)

    # Custom tight_layout to handle suptitle and legend spacing
    fig.tight_layout(rect=[0, 0.05, 1, 0.95])
    path = os.path.join(OUT_DIR, "forecast_overlay.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Overlay plot saved: {path}")

    # 2. Individual Plots
    for sid in chosen:
        fig_ind, ax_ind = plt.subplots(figsize=(10, 5))
        plot_single_stay(ax_ind, sid, stay_map[sid], is_individual=True)
        plt.tight_layout()
        path_ind = os.path.join(patient_dir, f"stay_{sid}.png")
        fig_ind.savefig(path_ind, dpi=120, bbox_inches="tight")
        plt.close(fig_ind)
    print(f"  Individual patient plots ({len(chosen)}) saved to {patient_dir}")


# ---------------------------------------------------------------------------
# 4. Horizon comparison (MAE per step h+1 ... h+12)
# ---------------------------------------------------------------------------
def plot_horizon_comparison():
    fig, ax = plt.subplots(figsize=(11, 5))
```

```python
    for model_name, mdir in MODEL_DIRS.items():
        color = MODEL_COLORS.get(model_name, "blue")
        yt = load_array(os.path.join(mdir, "y_true_test.npy"))
        yp = load_array(os.path.join(mdir, "preds_test.npy"))
        if yt is not None and yp is not None:
            mask = ~np.isnan(yt) & ~np.isnan(yp)
            mae_per_step = []
            for h in range(yt.shape[1]):
                m = mask[:, h]
                if np.any(m):
                    mae_per_step.append(np.mean(np.abs(yt[m, h] - yp[m, h])))
                else:
                    mae_per_step.append(np.nan)
            ax.plot(np.arange(1, len(mae_per_step) + 1), mae_per_step,
                    marker="o", color=color, linewidth=2, label=model_name)

    ax.set_xlabel("Forecast step (hours ahead)", fontsize=11)
    ax.set_ylabel("Mean Absolute Error (mmHg)", fontsize=11)
    ax.set_title("MAP Forecast Error by Horizon Step\n"
                 "(error should generally increase with longer horizons)",
                 fontsize=12, fontweight="bold")
    ax.set_xticks(range(1, PREDICT_LEN + 1))
    ax.legend(fontsize=9)
    ax.yaxis.grid(True, linestyle="--", alpha=0.6)
    ax.set_axisbelow(True)
    plt.tight_layout()
    path = os.path.join(OUT_DIR, "horizon_comparison.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"  Horizon comparison saved: {path}")


# ---------------------------------------------------------------------------
# 5. Error distribution
# ---------------------------------------------------------------------------
def plot_error_distributions():
    plot_data, labels, colors = [], [], []

    # All models
    for model_name in ["Prophet", "LSTM", "TFT/GRU", "Ensemble"]:
        mdir = MODEL_DIRS.get(model_name)
        color = MODEL_COLORS.get(model_name, "blue")
        if not mdir: continue

        yt = load_array(os.path.join(mdir, "y_true_test.npy"))
        yp = load_array(os.path.join(mdir, "preds_test.npy"))
        if yt is not None and yp is not None:
            # Drop NaNs
```

```
            errors = np.abs(yt - yp).mean(axis=1)
            valid_errors = errors[~np.isnan(errors)]
            if len(valid_errors) > 0:
                plot_data.append(valid_errors)
                labels.append(model_name)
                colors.append(color)

    if not plot_data: return

    fig, ax = plt.subplots(figsize=(10, 6))
    bp = ax.boxplot(plot_data, patch_artist=True, notch=True,
                    medianprops=dict(color="white", linewidth=2),
                    boxprops=dict(edgecolor="black", alpha=0.8),
                    whiskerprops=dict(linestyle="--", color="#666"),
                    flierprops=dict(marker="o", markerfacecolor="red", alpha=0.3, markersize=4))

    for patch, color in zip(bp["boxes"], colors):
        patch.set_facecolor(color)

    ax.set_xticks(range(1, len(labels) + 1))
    ax.set_xticklabels(labels, fontsize=11, fontweight="bold")
    ax.set_ylabel("Mean Absolute Error per window (mmHg)", fontsize=10)
    ax.set_title("MAP Prediction Error Variance -- Clinical Distribution", fontsize=14, fontweight="bold")
    ax.grid(True, axis="y", linestyle="--", alpha=0.3)
    ax.set_axisbelow(True)

    plt.tight_layout()
    path = os.path.join(OUT_DIR, "error_distribution.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"  Error distribution saved: {path}")

# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    print("=" * 60)
    print("  Evaluation & Reporting -- MAP Target")
    print("=" * 60)

    df      = pd.read_csv(os.path.join(DATA_DIR, "long_format.csv"))
    df_test = df[df["split"] == "test"].copy()

    print("[1/5] Leaderboard ...")
    lb = build_leaderboard()
    print("[2/5] Leaderboard plot ...")
    plot_leaderboard(lb)
```

```
    print("[3/5] Forecast overlay ...")
    plot_forecast_overlay(df_test)
    print("[4/5] Horizon comparison ...")
    plot_horizon_comparison()
    print("[5/5] Error distribution ...")
    plot_error_distributions()
    write_summary(lb)
    print("\nAll results saved to outputs/results/")


if __name__ == "__main__":
    main()
```

A.7 Step VII: Multi-Model Predictive Ensemble

Module 07: Multi-Model Weighted Predictive Ensemble

```
"""
07_model_ensemble.py
====================
Ensemble of LSTM and TFT/GRU models.
Weights are optimized (or simple average) to maximize R2.
Outputs -> outputs/ensemble/:
  metrics.csv
  preds_test.npy
  forecast_trajectories.png
"""

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error,
    mean_squared_error, r2_score

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(
    __file__)))
OUT_LSTM = os.path.join(BASE_DIR, "outputs", "lstm")
OUT_TFT  = os.path.join(BASE_DIR, "outputs", "tft")
OUT_DIR  = os.path.join(BASE_DIR, "outputs", "ensemble")
os.makedirs(OUT_DIR, exist_ok=True)

def load_array(path):
    return np.load(path) if os.path.exists(path) else None

def compute_metrics(y_true, y_pred):
    yt = y_true.flatten()
    yp = y_pred.flatten()
    mask = ~np.isnan(yt) & ~np.isnan(yp)
    yt, yp = yt[mask], yp[mask]

    mae = mean_absolute_error(yt, yp)
    rmse = np.sqrt(mean_squared_error(yt, yp))
    r2 = r2_score(yt, yp)
    mape = np.mean(np.abs((yt - yp) / yt)) * 100
```

```python
    return {"MAE": mae, "RMSE": rmse, "R2": r2, "MAPE": mape}

def main():
    print("=" * 60)
    print("  Model: Multi-Model Ensemble (LSTM + GRU)")
    print("=" * 60)

    y_true = load_array(os.path.join(OUT_LSTM, "y_true_test.npy"))
    p_lstm = load_array(os.path.join(OUT_LSTM, "preds_test.npy"))
    p_tft  = load_array(os.path.join(OUT_TFT,  "preds_test.npy"))

    if y_true is None or p_lstm is None or p_tft is None:
        print("ERROR: Missing prediction files. Run LSTM and TFT models first.")
        return

    # Option 1: Simple average
    # Option 2: Weighted average (LSTM performed slightly better, so 0.6 / 0.4)
    w_lstm = 0.6
    w_tft  = 0.4
    p_ens  = (w_lstm * p_lstm) + (w_tft * p_tft)

    metrics = compute_metrics(y_true, p_ens)
    print("\n-- Ensemble Metrics (MAP, test set) --")
    for k, v in metrics.items():
        print(f"  {k:6s}: {v:.4f}")

    # Save results
    np.save(os.path.join(OUT_DIR, "preds_test.npy"), p_ens)
    pd.DataFrame([metrics]).to_csv(os.path.join(OUT_DIR, "metrics.csv"), index=False)

    # Plot sample
    plt.figure(figsize=(12, 6))
    for i in range(min(3, len(y_true))):
        plt.subplot(3, 1, i+1)
        plt.plot(y_true[i], "k--", label="Observed")
        plt.plot(p_lstm[i], "b:", alpha=0.5, label="LSTM")
        plt.plot(p_tft[i], "g:", alpha=0.5, label="GRU")
        plt.plot(p_ens[i], "r-", linewidth=2, label="Ensemble")
        plt.legend(fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "forecast_trajectories.png"), dpi=150)
    print(f"\nEnsemble complete. Results saved to {OUT_DIR}")

if __name__ == "__main__":
    main()
```